

ENGE817: STEM Research Methods

Research Proposal

**The Systematic Extraction of Sensitive Information from
Embedded Linux Firmware Images Using Static Analysis Techniques**

Mohammad Mosfaqur Rahman

Student ID: 24271913

Master of Cyber Security and Digital Forensics

Auckland University of Technology

Supervisor: Dr Alastair Nisbet

Semester 1, 2026

AI tools, specifically ChatGPT and Microsoft Copilot, were utilised solely for brainstorming and idea generation. Grammarly, Microsoft Word and LLM-assisted spell check were employed for grammar, spelling and proofreading verification. Whilst these tools provided supporting assistance during preparation, all substantive writing, analysis and final content represent entirely the author's own work.

Table of Contents

List of Figures	iv
List of Tables	v
List of Acronyms	vi
Abstract.....	vii
1. Introduction.....	1
2. Literature Review.....	3
2.1. The scale and persistence of firmware security weaknesses	3
2.2. Credential extraction and hardcoded secret detection	4
2.3. Cryptographic key material and certificate exposure	5
2.4. Binary analysis for unsafe function detection	6
2.5. The gap in existing literature	7
3. Research Questions and Scoping	8
3.1. Research questions	8
3.2. Scope	9
4. Research Design.....	10
4.1. Overview of the seven-stage design	10
4.2. The fw-extract.sh automation script	11
4.3. Stage 1: firmware acquisition and filesystem extraction	12
4.4. Stage 2: credential and secret scanning	12
4.5. Stage 3: cryptographic key material extraction and classification	13
4.6. Stage 4: password hash extraction and cracking feasibility	14
4.7. Stage 5: binary static analysis.....	15
4.8. Stage 6: debug interface enumeration	16
4.9. Stage 7: results synthesis and taxonomy construction.....	16
4.10. Strengths, limitations, and alternative approaches	17
4.11. Ethical considerations.....	17
5. Application of the Research Method.....	19
5.1. Stage 1 pilot: firmware acquisition and filesystem extraction.....	20
5.2. Stage 2 pilot: credential and secret scanning	22
5.3. Stage 3 pilot: cryptographic key material extraction and classification	25
5.4. Stage 4 pilot: password hash cracking feasibility.....	27

5.5.	Relationship to research questions	29
6.	Evaluation of the Research Method	31
6.1.	Performance of the research design.....	31
6.2.	Critical reflection on the pilot study	31
6.3.	Findings in the context of the literature.....	32
7.	Conclusion.....	34
7.1.	Limitations.....	34
7.2.	Directions for further research.....	35
	References.....	38
	Appendix A - fw-extract.sh (Bash script).....	41
	Appendix B - Sample REPORT.md output (Askey 9361 pilot study)	48

List of Figures

Figure 4.1 Seven-stage static analysis research design.	10
Figure 4.2 Stage 3 manual key search: find and grep commands.	14
Figure 5.1 Alcatel-Lucent Askey 9361 3G Femtocell: Top View and Rear Panel.	19
Figure 5.2 Askey 9361 PCB: Front and Rear Sides.	20
Figure 5.3 SHA-256 Verification and MIME Type Identification of BSR-04.03.70.a.2.aky.tgz.	20
Figure 5.4 fw-extract.sh Output: SquashFS Identified, 4,975 Files Extracted.	21
Figure 5.5 The binwalk Signature Scan: SquashFS 4.0 Root Filesystem Identified.	21
Figure 5.6 The unsquashfs Extraction: 1,427 Files, 236 Directories, 317 Symlinks.	22
Figure 5.7 Stage 2, Step 1: Three UID 0 Accounts Identified in /etc/passwd.	22
Figure 5.8 Stage 2, Step 2: MD5-crypt (\$1\$) Hashes in /etc/shadow.dir/shadow.	23
Figure 5.9 Stage 2, Step 4: RADIUS_SECRET in /etc/init.d/radius, Commented Out but Present. ..	23
Figure 5.10 Stage 2, Step 5: fbsrpass TR-069 Credential Recovered from fpu1.vx via Strings Index.	25
Figure 5.11 Stage 3: dummy.key Confirmed as RSA 2048-bit Plaintext by openssl.	26
Figure 5.12 Stage 3: id_dsa confirmed as RSA 1024-bit plaintext; key length considered cryptographically weak.	26
Figure 5.13 The hashcat Mode 500 Output: rockyou.txt Exhausted Against Four MD5-crypt Hashes.	28

List of Tables

Table 4.1 Research question to method alignment.....	11
Table 5.1 Stage 2 pilot findings: credential and secret scanning results for Askey 9361 firmware BSR-04.03.70.a.2.aky.tgz.	24
Table 5.2 Stage 3 pilot findings: cryptographic key material classification for Askey 9361 firmware BSR04.03.70.a.2.aky.tgz.....	27
Table 5.3 Stage 4 Pilot Findings: Hash Cracking Feasibility for Askey 9361 Firmware.	29
Table 5.4 Pilot Study Results Mapped to Research Questions.....	30

List of Acronyms

API	Application Programming Interface
ARM	Advanced RISC Machine
ARM64	64-bit ARM architecture (ARMv8 instruction set)
CA	Certificate Authority
CPIO	Copy In/Out archive format
CVE	Common Vulnerabilities and Exposures
CWE	Common Weakness Enumeration
DER	Distinguished Encoding Rules
DSA	Digital Signature Algorithm
ECB	Electronic Codebook
ECDSA	Elliptic Curve Digital Signature Algorithm
Ed25519	Edwards-curve Digital Signature Algorithm (Curve25519)
ELF	Executable and Linkable Format
GPU	Graphics Processing Unit
HSPA+	High Speed Packet Access Plus
IoT	Internet of Things
IPsec	Internet Protocol Security
ISA	Instruction Set Architecture
IV	Initialisation Vector
JFFS2	Journalling Flash File System version 2
JTAG	Joint Test Action Group
LTE	Long-Term Evolution
MAC	Media Access Control
MD5	Message Digest Algorithm 5
MIME	Multipurpose Internet Mail Extensions
MIPS	Microprocessor without Interlocked Pipeline Stages
MQTT	Message Queuing Telemetry Transport
NIST	National Institute of Standards and Technology
OWASP	Open Worldwide Application Security Project
PCB	Printed Circuit Board
PEM	Privacy Enhanced Mail
PPK	PuTTY Private Key
PSK	Pre-Shared Key
QEMU	Quick EMUlator
RADIUS	Remote Authentication Dial-In User Service
RSA	Rivest-Shamir-Adleman
SFTP	SSH File Transfer Protocol
SHA	Secure Hash Algorithm
SoC	System on a Chip
SquashFS	Squashed File System
SSH	Secure Shell
TCF	Target Communication Framework
TR-069	Technical Report 069 (CPE WAN Management Protocol)
UFO	Unified Firmware Object (extraction tool)
UID	User Identifier
WAN	Wide Area Network
W-CDMA	Wideband Code Division Multiple Access

Abstract

Production embedded Linux firmware routinely contains hardcoded credentials, plaintext cryptographic key material, password hashes, unsafe function call sites and exposed debug interfaces, yet no framework in the reviewed literature appears to measure detection rates across these 5 categories in one pipeline. This research proposes a 7-stage ARM64 static analysis design automated through fw-extract.sh. A pilot on an Askey 9361 MIPS femtocell image yielded 9 credential and secret findings; all 9 were detected automatically, a rootfs-only pass detected 8, and both private keys were unencrypted. The full study applies all 7 stages, including Ghidra binary analysis, to 5 ARM64 production images.

Keywords: firmware static analysis; embedded Linux security; hardcoded credentials; cryptographic key extraction; binary vulnerability detection

1. Introduction

Embedded Linux firmware deployed across telecommunications infrastructure, consumer networking equipment and industrial control systems routinely contains sensitive material that tends to persist undetected, in large part because no unified extraction framework spans every such category in a systematic way. These images frequently carry hardcoded credentials, plaintext cryptographic keys and exposed debug interfaces, generally stored in predictable locations within the filesystem. Costin et al. (2014) demonstrated the scale of this problem through automated analysis of 32,356 firmware images drawn from manufacturer websites, extracting 109 private RSA keys and 100 distinct password hashes across 27 vendors. This evidence indicates that sensitive material is a significant, systemic pattern across the embedded device ecosystem, rather than an isolated finding in particular products.

The operational consequences of this exposure are considerable and, in practice, readily measurable. In 2016, the Mirai botnet infected 64,500 devices within its first 20 hours and reached a steady-state population of some 200,000 to 300,000 concurrently infected devices (Antonakakis et al., 2017). That same exploitation vector, specifically hardcoded default credentials, can be detected through static firmware analysis well before a device is ever deployed. Whilst the threat is both measurable and readily detectable, it nonetheless remains widely undetected at scale, which indicates that the deficiency lies in the detection tooling rather than in the threat itself.

Existing firmware security tools generally address individual vulnerability categories in isolation. Each tends to target password hashes, cryptographic key material or unsafe function call sites separately, and none spans all 5 categories together. Further, the precision of automated scanning, measured as the proportion of genuine findings among all automated hits, has not been reported for any firmware vulnerability category in the reviewed literature. From a single analysis pass, a security assessor therefore cannot readily determine whether a firmware image has been examined for every class of sensitive material, nor how much a given tool is likely to miss. The reviewed literature does not report automated detection rates measured across all 5 categories at once. This is the knowledge gap that the present research addresses. The research problem is the absence of a systematic, multi-category extraction framework for embedded Linux firmware that produces measurable and reproducible outputs.

The proposed research design is structured to produce exactly those outputs across the full set of 5 categories. The design consists of 7 sequential stages and targets ARM64 embedded Linux firmware in the full study. ARM64 denotes the 64-bit ARMv8 instruction set architecture (ISA), which is the dominant platform in contemporary embedded Linux devices. Stage 1 extracts the firmware filesystem using binwalk, following the approach established in the literature. Stages 2 through 4 then measure the credential, key material and hash categories that existing studies have generally examined only in isolation. Stage 5 provides the binary analysis whose detection completeness no study in the

reviewed literature has reported, whilst stage 6 enumerates the debug interfaces identified as a gap by Vishwakarma and Lee (2018). Stage 7 synthesises all 5 categories into a single and reproducible taxonomy.

The pilot study applies stages 1 through 4 to a MIPS-based device, and this is appropriate within the research design. Stages 1 through 4 rely on text pattern matching, cryptographic file validation and dictionary-based hash cracking, none of which depends in any meaningful way on the target instruction set architecture. Stage 5, which disassembles Executable and Linkable Format (ELF) binaries using Ghidra, is the only architecture-specific stage, and it is reserved for ARM64 devices in the full study. The coverage spans filesystem extraction, credential scanning, cryptographic key classification, password hash cracking feasibility, unsafe function usage detection, debug interface enumeration and results synthesis. Stages 1 through 4 are largely automated by `fw-extract.sh`, a Bash script developed for this research around open-source command-line tools. Additional tools include `binwalk`, `openssl`, `hashcat`, `Ghidra`, `strings` and `objdump`. The quantitative outputs include detection rates, plaintext key proportions, hash cracking feasibility measurements and unsafe function call site counts. Each term used across these stages is defined below.

Cryptographic key material refers to private keys, certificates and related artefacts stored in PEM, DER or PPK format. An unsafe function is a C library function whose misuse commonly produces exploitable vulnerabilities, including `system()`, `popen()`, `execve()`, `strcpy()` and `sprintf()`. JTAG ports and telnet consoles are the most common debug interfaces, providing privileged diagnostic access that firmware developers intend for laboratory use rather than production deployment.

This proposal is organised as follows. Chapter 2 reviews the literature on firmware analysis, credential exposure, cryptographic misuse and binary vulnerability detection. Chapter 3 states the 3 research questions derived from that review and defines the study scope. Chapter 4 describes the 7-stage research design in sufficient detail for replication by an independent analyst. Chapter 5 presents the pilot study findings from stages 1 through 4, applied to an Alcatel-Lucent Askey 9361 3G femtocell firmware image running on MIPS architecture. Chapter 6 evaluates the research design against the pilot evidence and reflects on the research process. Chapter 7 states the study's contributions, identifies the limitations of the research design and recommends directions for further research.

2. Literature Review

2.1. The scale and persistence of firmware security weaknesses

Firmware security weaknesses are distributed across production devices at a scale that, in practice, largely precludes manual inspection as a primary detection strategy. Costin et al. (2014) demonstrated this through automated analysis of 32,356 firmware images, extracting 109 private RSA keys from 428 images and 100 distinct password hashes from 681 images across 27 vendors. Plaintext passwords were recovered for 58 of those hashes, appearing across 538 images. Linux was present in some 86% of the analysed images, whilst ARM architecture accounted for approximately 63%. This pattern indicates that sensitive material is distributed at a density that demands an automated extraction pipeline rather than case-by-case review, and the correlation technique used in that study propagated a vulnerability identified in one image to similar images from other vendors. Each discovery therefore tends to apply across a considerably larger installed base than the single device under analysis.

The survey literature confirms that the problem spans multiple vulnerability categories simultaneously. Bakhshi et al. (2024) classified IoT firmware vulnerabilities across 8 influencing factor categories: system properties, access controls, hardware and software re-use, network interfacing, image management, user awareness, regulatory compliance and adversarial vectors. Each category is generally addressed by a different subset of the available tooling, and no single tool in the reviewed literature addresses all 8. The OWASP IoT Security Team (2018) ranked weak, guessable or hardcoded passwords first among IoT security issues, with insecure network services, insecure ecosystem interfaces and a lack of secure update mechanisms following in the ranking. Ul Haq et al. (2023) reviewed a number of firmware security frameworks and found that most suffer from poor scalability, high false positive or negative rates and limited cross-architecture support. This gap in tool coverage means that an assessor working with any single tool will miss a significant portion of the sensitive material present in the image under analysis.

Static analysis tends to be the dominant approach in the literature, largely because dynamic analysis is not feasible for most firmware. Nadir et al. (2022) reviewed the principal firmware analysis techniques and found that most solutions in the literature rely on static methods, because tight hardware dependencies tend to prevent successful emulation of the majority of production firmware images. This combination of constraints establishes static analysis as the most applicable approach to firmware security assessment. How reliably that analysis finds what is actually present then becomes the central and largely unanswered question. Zhou et al. (2023) tested approximately 6,000 firmware images and found that only about half had extractable filesystems, whilst a substantially smaller proportion could be fully emulated. Tien et al. (2018) found that even among extraction tools, filesystem recovery rates vary materially: UFO achieved 96% against binwalk's 87% across 90 test images, and material left in an unextracted filesystem is simply unavailable for any subsequent analysis pass. Measuring automated

detection rates against a manual baseline, across all 5 categories at once, is how this research design seeks to address that gap.

2.2. Credential extraction and hardcoded secret detection

Hardcoded credential exposure has been studied through purpose-built tooling and manual reverse engineering, but the results are typically category-specific. Shen et al. (2021) analysed firmware images for open-source component versions, hardcoded strings and weak passwords. The open-source component versions in the analysed images lagged, on average, some 5.7 years behind current releases. Whilst hardcoded credentials were a consistent finding, they were generally treated as indicators of insecure practice rather than as a structured category requiring systematic enumeration, and the study did not measure how much automated scanning missed relative to a manual baseline. Ahmed et al. (2020) applied static analysis to 1,230 firmware images to identify Mirai-exploitable vulnerabilities, finding 193 susceptible images, or approximately 15.7% of the total. This approach detected telnet-accessible default credentials but did not measure what proportion of credential items a manual inspection would likely have found in addition.

FIRMPASS, developed by Huang et al. (2025), provides a more systematic approach to password-related analysis. FIRMPASS was applied to 615 Linux-based IoT firmware images across 4 password management weakness types and reported 82 candidate cases, of which 67 were confirmed as genuine vulnerability instances, yielding a precision of 81.7%. The paper does not report recall, that is, the proportion of password-related vulnerabilities actually present in the 615 images that were detected, which leaves the false negative rate unquantified. This unquantified gap is a characteristic limitation of all single-category automated tools. The reviewed literature does not quantify detection completeness across a broader set of sensitive information categories, which establishes the need for a multi-category comparison framework. Further, Nadir et al. (2019) developed an auditing framework applying binwalk and Firmwalker to an IP camera and identified hardcoded credentials, default SSH configurations and insecure service settings. Whilst that framework addressed a wider range of categories than FIRMPASS, it was applied to a single device and did not report how accurately it identified all credential material present.

Femtocell-specific research demonstrates that credential exposure in telecommunications infrastructure firmware is particularly acute. Borgaonkar et al. (2011) achieved root access on a femtocell by recovering an MD5-hashed root password from an unencrypted recovery filesystem and enabling root login over SSH through a configuration modification. Yang et al. (2026) assessed 4 production 4G LTE femtocells and found that all 4 exhibited credential extraction vulnerabilities and predictable credential patterns, whilst 3 of the 4 devices also exposed management services over the WAN interface. That exposure allowed remote exploitation using the recovered credentials. Begian (2023) applied static analysis tools to 5 5G femtocell firmware images and identified thousands of CVEs

across all samples. Golde et al. (2012) demonstrated that a rogue femtocell exploiting extracted credential material can function as a trusted network node and intercept mobile device communications without requiring a protocol downgrade. A private key stored in plaintext within a firmware image is, in effect, immediately usable for such an attack. No password cracking, hardware access or network interception is required beyond simply obtaining a copy of the firmware. This body of work confirms that the credential extraction problem is present in the specific firmware class examined in this research proposal, and that its consequences may extend to the network level. Across all 3 studies, however, detection was reported as a binary outcome: credentials were either found or not found. None measured what proportion of credential material a given tool misses, which is precisely the quantity that determines whether a firmware image can reasonably be considered audited. This absence of a detection rate baseline is the deficiency that research question 1 directly addresses.

2.3. Cryptographic key material and certificate exposure

Cryptographic misuse in IoT firmware encompasses both API misuse during encryption operations and the plaintext storage of key material in the filesystem. These are distinct and significant problems, and they tend to require rather different detection approaches. Zhang et al. (2019) developed CRYPTOREX and applied it to 521 firmware images, identifying 679 cryptographic misuse bugs across 126 images. The bugs spanned incorrect key derivation, IV reuse and ECB mode encryption. Whilst CRYPTOREX detected cryptographic key material in firmware images, it focused on API misuse patterns rather than on whether the stored key files were themselves protected by a passphrase. Further, Costin et al. (2014) extracted 41 self-signed SSL certificates together with their corresponding private RSA keys from firmware images. They then correlated these certificates against the ZMap HTTPS survey database and found approximately 35,000 active devices sharing the extracted keys. If for example a single embedded key terminates the administrative interface on every unit of a model, then recovering it from one published image compromises the entire fleet at once. Plaintext key storage in a firmware image therefore exposes the entire installed base of a device model rather than only a single unit, since an attacker with filesystem access requires no further decryption step: the key material is immediately usable.

Whether firmware-resident key material is stored in plaintext or in an encrypted form is a measurement that the existing literature does not report. Li et al. (2022) developed a firmware fingerprinting approach achieving 91% precision and 90% recall at the fine-grained version level. Images from the same device family tend to share structural characteristics, including filesystem layout and key file locations, so a key file found in one image from a family is likely to be present in all images from that family. Janzen et al. (2024) assessed an O-RAN radio unit and found that it retained default credentials alongside unauthenticated service access through an exposed TCF agent and MQTT server, whilst root command execution was achieved with Ethernet access only. Neither study, however,

reports what proportion of key material in a firmware image is stored in plaintext. This proportion in considerable part determines how large a fraction of the cryptographic risk an attacker with filesystem access can immediately exploit. No reviewed study reports this measurement for production firmware, which leaves the substantial scale of direct key exposure essentially unquantified. Research question 2 is structured to produce exactly that measurement for a defined ARM64 firmware sample.

2.4. Binary analysis for unsafe function detection

Binary-level static analysis of firmware has been studied through taint analysis, function signature matching and disassembly-based pattern detection. Qasem et al. (2021) produced a layered taxonomy of automatic vulnerability detection mechanisms and identified 12 challenges specific to binary analysis. These include information loss during compilation, architecture-specific disassembly constraints and the absence of source-level context. This substantial set of challenges generally bounds what automated static analysis can reliably detect. Static analysis can produce false positives where the call context does not confirm exploitability, and equally false negatives where indirect function dispatch conceals an exploitable usage pattern. No source in the reviewed literature quantifies the proportion of unsafe function call sites concealed by indirect dispatch in embedded Linux firmware.

Several studies provide converging evidence that unsafe function exposure is both prevalent and structurally concentrated, yet none measures how much automated tooling detects. He et al. (2021) detected 300 security problem cases across 6 categories using hybrid static and dynamic analysis, with unsafe dependency library calls and anomalous process execution accounting for the 2 largest proportions at 26.33% and 21.67% respectively. These 2 classes are therefore numerically dominant. Al-Zuraiqi and Greer (2025) applied the RMMIDL framework to 1,520 firmware images without emulation and identified CWE-134 format string vulnerabilities in 10.07% of samples, with CWE-782 exposed debugging tools in a further 10.07%. Static binary inspection alone can therefore surface a material proportion of the exposure. Liu et al. (2017) further showed that unsafe function calls are concentrated in vendor-added code rather than open-source system libraries, which means that application binaries warrant priority and that the distribution of exposure is non-uniform across the filesystem. Zhao et al. (2022) demonstrated through the FirmSec framework that third-party components carry known N-day vulnerabilities into any image that includes them. This adds a component-inheritance dimension: if for example a firmware image uses an outdated version of OpenSSL or BusyBox, it inherits all unpatched CVEs from that component. This body of evidence shows that unsafe function exposure is prevalent (He et al., 2021; Al-Zuraiqi & Greer, 2025), concentrated in vendor code (Liu et al., 2017) and compounded by component inheritance (Zhao et al., 2022). None of the reviewed studies, however, measures the proportion of that exposure that automated binary analysis actually identifies relative to a full disassembly review. This absence limits the practical

value of binary analysis findings for assessors, since without a detection rate they cannot judge how much a given tool misses. Research question 3 is structured to produce that measurement.

2.5. The gap in existing literature

Across the reviewed studies, no single research design spans all 5 sensitive information categories, that is, credentials, cryptographic key material, password hashes, unsafe function call sites and debug interfaces, with automated detection rate measurement relative to a manual baseline. The survey-level reviews broadly confirm that this gap reflects a systemic deficiency in the field rather than a shortcoming in any single study.

What the literature has established is substantial: sensitive material is present in production firmware at significant scale, specific vulnerability classes are reliably detectable through static analysis, and individual tools can typically achieve a measurable accuracy within their own category (Costin et al., 2014; Huang et al., 2025; Al-Zuraiqi & Greer, 2025). However, the measurements that remain absent are how much each tool misses across the full set of categories and what proportion of key material is stored in recoverable plaintext. A single research design spanning all 5 categories with quantitative detection rate measurement does not appear, on the evidence reviewed, to exist.

This study is necessary because those measurements do not yet exist and because firmware security assessors currently lack a repeatable procedure for producing them. No reviewed study produces these measurements at once across all 5 categories, which is the specific gap the present research addresses. How, then, should a practitioner judge whether a firmware image has been adequately examined? The answer, at present, is that no established procedure makes that judgement possible.

3. Research Questions and Scoping

Three gaps in the reviewed literature motivate the research questions that follow. The 5 categories of sensitive material examined in this research are credentials, cryptographic key material, password hashes, unsafe function call sites and debug interfaces. First, no study identified in the reviewed literature has systematically measured how effectively static analysis identifies and quantifies sensitive information across all 5 vulnerability categories in embedded Linux firmware, which leaves assessors with little basis for judging the coverage or reliability of automated tooling. Second, none reports what proportion of cryptographic key material in production firmware is stored without passphrase protection, leaving the substantial scale of immediate key exposure essentially unmeasured. Third, none measures how accurately open-source static analysis tools identify unsafe function call sites in ARM64 binaries relative to a full disassembly review, so an assessor cannot readily judge how much automated binary analysis misses. Each of these gaps motivates one research question.

3.1. Research questions

RQ1: How effectively does static analysis extract sensitive information across multiple vulnerability categories in embedded Linux firmware images?

RQ1 addresses the core measurement problem identified in the literature review. Answering this question should produce the detection rate and precision data that firmware security assessors currently lack when evaluating the completeness of automated tooling.

RQ2: What proportion of cryptographic key material in embedded Linux firmware is stored without passphrase protection?

RQ2 targets a specific and significant subset of the extraction problem. If for example an SSH host key sits in plaintext in `/etc/ssh`, then any party holding the published firmware holds that key directly, yet the reviewed literature does not report what proportion of firmware-resident key material is stored without passphrase protection.

RQ3: To what extent do open-source static analysis tools identify unsafe function call sites in ARM64 ELF binaries extracted from production firmware?

RQ3 addresses the binary analysis dimension that static string matching and filesystem inspection cannot reliably reach. Unsafe function call sites require disassembly to identify, and this detection completeness measurement is absent from the reviewed literature for ARM64 production firmware.

3.2. Scope

This research applies static analysis to embedded Linux firmware images compiled for the ARM64 ISA. ARM64 is selected largely because ARM architecture accounted for approximately 63% of the firmware images in the large-scale analysis by Costin et al. (2014), and because it is the dominant variant in contemporary telecommunications infrastructure. Stages 1 through 4 operate on filesystem content and hash values rather than on machine code, and are therefore architecture-agnostic. The pilot applies these stages to an Alcatel-Lucent Askey 9361 device running a MIPS-based firmware image, an older architecture typical of legacy 3G femtocell deployments, whilst the full study extends the stage 5 binary analysis to ARM64 production firmware.

The exclusions are those approaches that require live device access, network interaction or physical hardware modification. Each exclusion is listed because it would otherwise be a plausible extension of the research scope. Dynamic analysis is excluded. Although it would in principle reveal runtime behaviour, Cheng et al. (2018) note that in a study of more than 6,500 firmware images, approximately 90% lack source code and cannot be dynamically analysed, owing to the absence of source code and hardware-specific peripheral dependencies. Wireless traffic interception is excluded because it requires network access to a running device. Hardware fault injection and TrustZone exploitation are excluded because they require physical device access and specialised instrumentation well beyond the defined toolset. Supply chain analysis is excluded because it requires access to build environments and version control histories that are unavailable from a firmware image alone. Dynamic instrumentation frameworks including Frida are excluded because they require a running process.

The pilot study applies stages 1 through 4 to a single Alcatel-Lucent Askey 9361 3G femtocell firmware image. The research design in Chapter 4 specifies how each stage is executed and how its outputs feed into the next.

This research produces 3 quantitative contributions: a detection rate and precision measurement across all 5 sensitive material categories against a cross-partition manual baseline (RQ1); a plaintext-to-encrypted key ratio for ARM64 production firmware (RQ2); and a lightweight tool detection rate against a Ghidra cross-reference baseline for unsafe function call sites in ARM64 ELF binaries (RQ3). These outputs therefore form a reproducible static analysis pipeline that firmware security assessors can apply to any extractable embedded Linux image.

4. Research Design

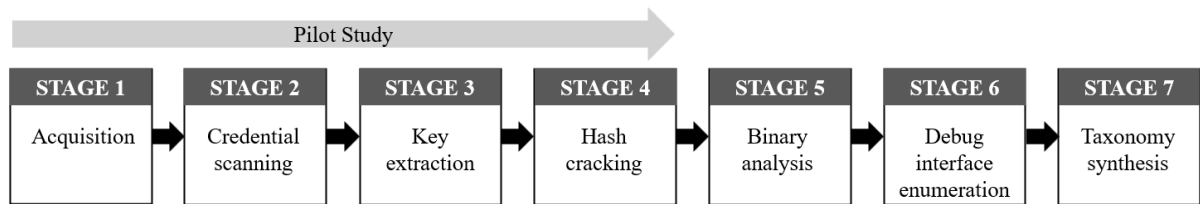
This research uses a quantitative and experimental design. The results are expressed as counts, proportions and detection rates, and these outputs are measurable, reproducible and comparable across firmware images. The design requires no dynamic analysis and no live device access, and a standard workstation is sufficient.

4.1. Overview of the seven-stage design

The research design consists of 7 sequential stages. Stages 1 through 3 address filesystem extraction and surface-level sensitive material identification, whilst the pilot study applies stages 1 through 4 to a single firmware image. Stages 4 through 6 address the deeper analysis requiring hash cracking, binary disassembly and debug service enumeration, and stage 7 synthesises all of the results into a structured taxonomy with detection rate measurements. Each stage produces a defined output that becomes the input to the next. In this way, an independent analyst can replicate the procedure using the same open-source toolchain on the same firmware image. Figure 4.1 illustrates the full design, with the tool and RQ linkages shown for each stage.

Figure 4.1

Seven-stage static analysis research design.



Note. The 7 stages progress from firmware extraction through credential scanning, key material classification, hash cracking, binary analysis, debug enumeration and taxonomy synthesis. Stages 1 through 4 are covered in the pilot study; stages 5 through 7 are reserved for the full study.

Table 4.1 maps each research question to the stages, tools and outputs that address it.

Table 4.1

Research question to method alignment.

RQ	Stages	Primary tools	Output metric
RQ1: Effective extraction across 5 categories	1-4, 6, 7	binwalk, fw-extract.sh, grep, strings, openssl, hashcat	Detection rate, precision and hash-cracking feasibility per category
RQ2: Proportion of key material stored in plaintext	1, 3	binwalk, openssl, puttygen	Plaintext-to-encrypted ratio
RQ3: Extent of unsafe function identification in ARM64 ELF binaries	1, 5	Ghidra, strings, objdump	Lightweight tool detection rate against Ghidra cross-reference baseline

Note. RQ3 is addressed only in the full study; the pilot applies stages 1-4.

4.2. The fw-extract.sh automation script

The Bash script *fw-extract.sh* was authored for this research to automate the multi-stage extraction and scanning workflow. The full script is provided in Appendix A. The script is built around open-source command-line tools and accepts a firmware image file and an output directory as arguments. The output tree includes extracted filesystems across all embedded partitions, a findings directory with 5 subdirectories (keys, certs, configs, credentials and hashes), a cross-partition strings index, a binwalk signature scan log and a REPORT.md summarising all results by category. A sample REPORT.md output from the pilot study is provided in Appendix B. No tool identified in the reviewed literature covers all 5 functions in one reproducible pipeline (Nadir et al., 2022; Ul Haq et al., 2023). These 5 functions are: cross-partition extraction, credential scanning, key validation, binary protection checking and debug interface enumeration. Every analysis step in *_fw-extract.sh_* is visible, auditable and re-runnable from a single command.

Nine sequential modules constitute the script, covering filesystem extraction with chain-of-custody recording, SSH key and certificate scanning, user account and password hash extraction, hardcoded credential detection across a cross-partition strings index, binary protection analysis and debug interface enumeration. Cross-partition coverage is the mechanism by which the script detects credentials compiled into non-rootfs binaries that a rootfs-restricted manual pass cannot reach. During the pilot study, the key classification module was found to report all identified key files, including public key files, under the "private keys" counter in the REPORT.md summary. This miscategorisation arose because the module applied the same find filter for all key-file extensions without distinguishing private from public keys in the summary tally. The module was corrected to apply `openssl rsa -text -noout` and `openssl ec -text -noout` to each candidate. Files parsed as private keys are counted under

private keys; files yielding only a public key parse are counted separately under public keys. The corrected module is reflected in the script provided in Appendix A.

4.3. Stage 1: firmware acquisition and filesystem extraction

Firmware images for the full study are sourced from manufacturer public download portals or publicly accessible firmware repositories. Images are selected according to 4 inclusion criteria. First, the image must be compiled for the ARM64 ISA, as confirmed by the ELF machine type header of at least one binary in the extracted filesystem. Second, the image must produce a successfully extracted root filesystem using binwalk version 2.4 and unsquashfs, without requiring manual intervention beyond the tools' standard capability. Third, the image must originate from a device category relevant to the research context: telecommunications infrastructure, consumer networking equipment or industrial control systems. Fourth, the image must be publicly available without authentication, to satisfy the lawful acquisition requirement in Section 4.11. To ensure vendor diversity and avoid findings that reflect the practices of a single manufacturer rather than the field, no more than one image per vendor is selected. Where a vendor offers multiple firmware lines, the most recent publicly available release from the primary product line is chosen. If an image fails the extractability criterion after acquisition, it is excluded and replaced by the next qualifying image from a different vendor, with the substitution recorded in the dataset description. Five images satisfying all criteria are selected. This number provides sufficient variation to assess taxonomy stability across device categories whilst remaining within the analytical capacity of a single researcher working without automated emulation infrastructure. The approach is consistent with prior framework studies operating on comparable small device sets (Nadir et al., 2019; Cheng et al., 2018). The generalisability limitation this imposes is discussed in Chapter 7.

The *fw-extract.sh* Bash script automates stage 1. It identifies the firmware archive MIME type and invokes binwalk version 2.4 with recursive extraction at a maximum depth of 5 levels. The unsquashfs tool is then applied to any SquashFS image identified during extraction. Compressed filesystem structures including SquashFS, JFFS2 and cramfs are identified and extracted automatically. binwalk is the most widely used open-source firmware extraction tool in the literature (Costin et al., 2014). It produces structured output for each identified filesystem type on any Linux analysis platform. Each successfully extracted partition is catalogued by type, size and path. The extracted filesystem root is the primary input for all subsequent stages.

4.4. Stage 2: credential and secret scanning

The credential scanning component of *_fw-extract.sh_* is applied to the extracted filesystem to identify credential material in plain-text configuration files, init scripts and shell scripts. The component applies regular expression patterns designed to match password assignment syntax, API key formats, pre-shared key declarations and bearer token strings. The regex pattern set targets credential categories

identified across large-scale firmware studies and IoT security frameworks (Costin et al., 2014; Nadir et al., 2019, 2022; OWASP IoT Security Team, 2018). File paths are searched in priority order: `/etc/passwd`, `/etc/shadow`, `/etc/ssh/ssh_config`, `init.d` scripts and application configuration files in `/app`, `/opt` and `/usr/local` are examined first, followed by a full recursive scan of the filesystem. The strings utility is also applied to all ELF binaries in the extracted filesystem, with output piped through the same regex patterns. This step captures credential material embedded in compiled binaries that a text-only file scan would not reach.

A manual analysis pass is conducted over the same filesystem independently, without reference to the automated scan results. The manual pass follows a 6-step structured procedure. Step 1 examines `/etc/passwd` for non-root accounts assigned UID 0, accounts with interactive shells and accounts with blank or trivially guessable passwords. Step 2 inspects `/etc/shadow` and any shadow file variants for hash format prefixes and for accounts with no password field. Step 3 reads all files in `/etc/ssh/` individually, including `ssh_config` for `PasswordAuthentication` and `PermitRootLogin` settings, and any `id_*` or `ssh_host_*` files for private key content. Step 4 reads every file in `/etc/init.d/`, `/etc/rc.d/` and `/etc/rcS.d/` in full for hardcoded credential strings, commented-out secrets and management server references. Step 5 applies strings manually to each ELF binary across all extracted partitions, not only the rootfs, searching output for password, username, secret, key and token patterns. Extending the manual pass to all partitions is essential because, as the pilot study demonstrated, credential material compiled into non-rootfs application binaries is not accessible to a rootfs-restricted manual inspection. Step 6 opens any configuration file identified by the `find` command with a `.conf`, `.cfg` or `.ini` extension and reads it for credential patterns not matched by the regex set. Items found in the manual pass but absent from the automated output are recorded as false negatives. Items flagged by the automated scan but not confirmed as genuine credentials are recorded as false positives. The automated detection rate for RQ1 is the ratio of automated detections to the total item count established by the cross-partition manual baseline.

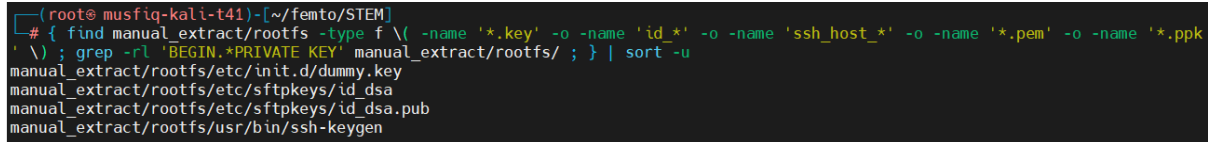
4.5. Stage 3: cryptographic key material extraction and classification

Manual key enumeration precedes automated scanning in stage 3 to provide an independent baseline against which automated detection completeness is measured. Three commands are executed in sequence before the script is run. First, `find` is applied recursively from the filesystem root with name filters matching common key file naming patterns: `*.key`, `*.pem`, `*.crt`, `*.cer`, `*.der`, `*.ppk`, `id_rsa`, `id_dsa`, `id_ecdsa`, `id_ed25519` and `ssh_host_*`. Second, `grep` is applied recursively to locate files containing PEM header strings. The patterns targeted are: `BEGIN RSA PRIVATE KEY`, `BEGIN EC PRIVATE KEY`, `BEGIN OPENSSH PRIVATE KEY`, `BEGIN PRIVATE KEY` and `PuTTY-User-Key-File`. Third, `file` is applied to all outputs to confirm content type, and any file identified by extension but not

confirmed as a key by `grep` is opened manually and inspected. The results of this manual search are recorded before the automated script is executed, as shown in Figure 4.2.

Figure 4.2

Stage 3 manual key search: find and grep commands.



```
(root@musfiq-kali-t41)~[~/femto/STEM]
# { find manual_extract/rootfs -type f \( -name '*.key' -o -name 'id_*' -o -name 'ssh_host_*' -o -name '*.pem' -o -name '*.ppk'
\ ) ; grep -rli 'BEGIN.*PRIVATE KEY' manual_extract/rootfs/ ; } | sort -u
manual_extract/rootfs/etc/init.d/dummy.key
manual_extract/rootfs/etc/sftpkeys/id_dsa
manual_extract/rootfs/etc/sftpkeys/id_dsa.pub
manual_extract/rootfs/usr/bin/ssh-keygen
```

Note. Commands executed before the automated scan to establish an independent key file baseline.

The automated scan then applies the same `find` and `grep` patterns programmatically and validates each candidate using `openssl`. A file that `openssl` parses without a passphrase prompt is classified as plaintext. A file that triggers a passphrase prompt or returns a bad decrypt error is classified as encrypted. Files `openssl` cannot parse are returned to manual inspection. The analyst applies `file` and `xxd` to determine whether the file is malformed, uses a non-standard encoding or requires `openssl asn1parse` for DER format. PPK files identified in either pass are validated using `puttygen` with the `--check` flag. The `puttygen` tool is the reference implementation for PuTTY key file validation and is available in standard Debian-based package repositories. The comparison between the manual search results and the automated scan output identifies any key files the script missed, which are recorded as false negatives for RQ2.

For each identified key file, the output records 4 fields: the file path, the key type (RSA, EC, Ed25519, DSA or pre-shared), the storage format (PEM, DER or PPK) and the plaintext or encrypted classification. Plaintext-to-encrypted ratios across the full key file inventory answer RQ2. The `openssl` utility is selected because it is the standard validation tool for PEM and DER format key material on Linux analysis platforms.

4.6. Stage 4: password hash extraction and cracking feasibility

Stage 4 targets the password hashes extracted from `/etc/shadow` and `/etc/passwd` files identified in stage 2. Format prefix identifies the algorithm: `$1$` is MD5-crypt, `$5$` is SHA256-crypt and `$6$` is SHA512-crypt. The full study targets SHA512-crypt hashes using `hashcat` in mode 1800. The `rockyou.txt` wordlist, containing 14 million passwords from the RockYou data breach, is the primary dictionary input. This is the standard benchmark dictionary for password cracking research (Melicher et al., 2016). Further, Costin et al. (2014) used John the Ripper with a custom dictionary built from common password lists and strings extracted from firmware, manuals and readme files.

For the full study, cracking is time-bounded at 24 hours per firmware image. Where the wordlist is fully exhausted before the 24-hour limit, the run is recorded as exhausted with the actual elapsed time.

The output is a table recording, for each hash: the format, account name, whether cracking succeeded within the time limit and the time to crack if successful. The full study applies 2 wordlists in sequence. The first is rockyou.txt as the general dictionary baseline; the second is a device-specific vocabulary constructed from device identifiers, vendor prefix strings and strings drawn from firmware documentation and release notes. This two-stage approach distinguishes general dictionary resistance from targeted attack feasibility. The hashcat tool is selected because it supports GPU acceleration and is widely regarded as the benchmark in published password cracking research. The 24-hour time bound produces a practical feasibility measurement rather than an exhaustive attack.

4.7. Stage 5: binary static analysis

Stage 5 answers RQ3 by enumerating unsafe function call sites in ARM64 ELF binaries and comparing automated tool coverage against full Ghidra disassembly. Binaries are identified using the file command applied recursively from the filesystem root. Application binaries in /bin, /sbin, /usr/bin and vendor-specific directories are processed before system libraries. This prioritisation follows the finding of Liu et al. (2017) that unsafe function calls are concentrated in vendor-added code. Zhao et al. (2022) further established that third-party components carry known N-day CVEs that propagate into any firmware image that includes them. Because both vendor-authored code and the third-party components it incorporates may contain exploitable function call patterns, application-layer binaries warrant priority in the analysis.

To bound the analysis within the capacity of a single analyst, a maximum of 30 ARM64 ELF application binaries per firmware image are selected for Ghidra disassembly, chosen in descending order of file size. Size-based selection prioritises the binaries most likely to contain substantial vendor-authored code. Where an image contains fewer than 30 qualifying application binaries, all are included. System libraries in /lib and /usr/lib are excluded from the Ghidra analysis unless a vendor-authored binary imports a function from one that is itself listed in the unsafe function set. Each binary is loaded into Ghidra and analysed within a 20-minute time budget per binary. This bound is set conservatively based on observed Ghidra auto-analysis times for MIPS binaries of comparable size during the pilot study; ARM64 binaries of similar file size are expected to fall within the same window. If auto-analysis does not complete within 20 minutes, the binary is excluded and a note is recorded.

Each selected binary is loaded into Ghidra. Ghidra is selected over radare2 and Binary Ninja because it is freely available under an Apache 2.0 licence and provides a scripting API for automated cross-reference analysis. Huang et al. (2025) applied Ghidra for cross-architectural binary decompilation in the FIRMPASS pipeline, demonstrating its suitability for automated firmware binary analysis of the kind required in stage 5. Cross-reference analysis enumerates all direct call sites of the following functions: system(), popen(), execve(), strcpy(), sprintf(), fprintf() and gets(). Indirect calls, where the function is invoked through a function pointer or dispatch table, are noted when visible in the

Ghidra disassembly. They are recorded as a separate count and excluded from the automated detection rate denominator, because neither strings nor objdump can identify them. This boundary is a known limitation of static binary analysis and is discussed in Section 4.10. As a lighter-weight complement, strings extracts printable character sequences from each binary and objdump is applied with the dynamic symbol table flag to identify imported unsafe functions from the symbol table. The proportion of direct unsafe function call sites found by strings and objdump relative to those found through full Ghidra cross-reference analysis contributes to answering RQ3. The difference in call site density between application binaries and system libraries also contributes to answering RQ3.

4.8. Stage 6: debug interface enumeration

Startup scripts, init.d files and systemd unit files are examined for references to local network services, hardcoded IP addresses and port numbers and debug protocol invocations including telnet, gdbserver and JTAG forwarding daemons. The examination applies grep recursively to the filesystem with patterns targeting common debug service indicators. Hardcoded IP address and port references are extracted using a regular expression matching dotted-decimal notation followed by a colon and a numeric port.

Each identified service is recorded with its binding address, port and the script or binary responsible for invocation. Services are classified by protocol type and network exposure. Vishwakarma and Lee (2018) documented JTAG as the primary means of accessing hardware-protected memory in embedded devices. Services bound to loopback addresses are recorded as locally exposed debug interfaces. Services bound to all interfaces (0.0.0.0) are recorded as network-exposed. Enumeration operates entirely through static inspection of configuration and script files. No network scanning or live device interaction is performed.

4.9. Stage 7: results synthesis and taxonomy construction

Outputs from stages 2 through 6 are consolidated into a structured taxonomy. For each sensitive information category, 5 fields are recorded: the total count from the cross-partition manual baseline; the count from automated tooling alone; the automated detection rate as a percentage; the automated precision as a percentage; and a qualitative risk classification. Precision is defined as the proportion of automated detections subsequently confirmed as genuine items by manual review. For the credential category, precision figures are already available from the pilot study; for cryptographic key material, password hashes, unsafe function call sites and debug interfaces, precision will be calculated from full-study results. The automated detection rate is defined as the number of items found by the automated script divided by the total number of unique items identified by the cross-partition manual baseline. The cross-partition manual pass, which covers all extracted partitions, is the ground truth for this calculation. An item found by both methods is counted once in the denominator. This definition ensures that the

detection rate measures automated coverage against a genuine census of sensitive material, rather than against a partial baseline restricted to a subset of the image.

Plaintext key proportions from stage 3, hash cracking results from stage 4 and unsafe function call site counts from stage 5 are incorporated as category-specific metrics. The completed taxonomy is compared against the vulnerability categories enumerated by Nadir et al. (2022), Bakhshi et al. (2024) and the OWASP IoT Top 10 (OWASP IoT Security Team, 2018) for coverage. Categories present in the prior literature but absent from this research design are recorded as limitations.

The detection rate, as used in this research, is a measure of auditing completeness rather than a measure of vulnerability prevalence. This distinction matters for how the results are interpreted. A detection rate measures the fraction of a known population of items that a given tool or procedure identifies. It therefore presupposes that the denominator, the total population of sensitive items present, has been established independently. In this design, that denominator is supplied by the cross-partition manual baseline.

4.10. Strengths, limitations, and alternative approaches

The principal strength of this research design is that every stage produces a quantitative output that can be reproduced by any analyst using the same open-source toolchain on the same firmware image. This design also avoids the emulation dependency that Cheng et al. (2018) identified as a significant constraint on the dynamic analysis of approximately 90% of production firmware, owing to the absence of source code and hardware-specific peripheral dependencies. Further, measuring automated detection rates relative to manual analysis provides a calibration metric that prior static analysis studies have not generally reported.

Dynamic firmware analysis using QEMU or FIRMADYNE is the primary alternative to this research design. For ARM64 in particular, emulation success rates tend to be further constrained by hardware peripheral dependencies. Static analysis is therefore the appropriate primary method for the firmware class and architecture under examination. Whilst dynamic analysis can identify the call sites reachable from network input at runtime, which static analysis cannot determine, distinguishing practically exploitable call sites from those requiring specific runtime preconditions is outside the scope of this static analysis design. The limitations of this design are discussed fully in Chapter 7.

4.11. Ethical considerations

This research does not involve human participants, live device access or active network interaction, and therefore does not require ethics committee approval. All firmware images are obtained from publicly accessible manufacturer download portals or equivalent public repositories and are used

solely for security research purposes consistent with the terms under which they are made available. No device is powered on, connected to a network or otherwise activated at any point during the study.

Credential material extracted from firmware images is used only to record its presence and classify its type; no credential is used to attempt access to any live system. All extracted credential material is redacted in any data shared with third parties or included in published outputs: password hashes are truncated to their format prefix, private key material is represented by file path and key type only, and bearer tokens or pre-shared keys are summarised by their detected pattern class rather than reproduced verbatim. This redaction policy ensures that the research outputs cannot be used to facilitate access to production systems.

The study follows a passive disclosure posture: findings are reported in aggregate without identifying specific live production systems. Where a finding appears to represent a systemic vulnerability affecting an identifiable active product line, the vendor will be notified in accordance with standard responsible disclosure practice before any detailed finding is published. This position is consistent with the research design's emphasis on measurable and reproducible outputs rather than on active exploitation.

5. Application of the Research Method

The pilot study applies stages 1 through 4 of the research design to the firmware of an Alcatel-Lucent Askey 9361 3G femtocell device, a low-power W-CDMA and HSPA+ home cell operating at 2100 MHz (Alcatel-Lucent, 2013). The firmware image, designated BSR04.03.70.a.2.aky.tgz, was obtained from a physically acquired device, whilst the full study will source its images from public manufacturer download portals, as specified in the research design. Physical acquisition is a permissible pilot variant where a public image is unavailable. Its compressed size is 120 MB and its SHA-256 digest is cbc566a4be3491a8f2ae0dc9df97a0ade4ef9e223af696415bdb463ff966b1be. This hash was recorded before analysis began so as to establish chain of custody over the image. The extracted filesystem identifies the processor architecture as MIPS, and the Askey 9361 therefore uses MIPS rather than ARM64. Stages 1 through 4 rely on text pattern matching, cryptographic file validation and dictionary-based hash cracking, none of which depends in any meaningful way on the instruction set architecture of the target device. MIPS is therefore an appropriate pilot architecture for these stages. Stage 5, which performs ELF disassembly in Ghidra, is the only architecture-specific stage and is applied to ARM64 devices in the full study only. Figures 5.1 and 5.2 show the physical device and PCB of the Askey 9361 used in the pilot study.

Figure 5.1

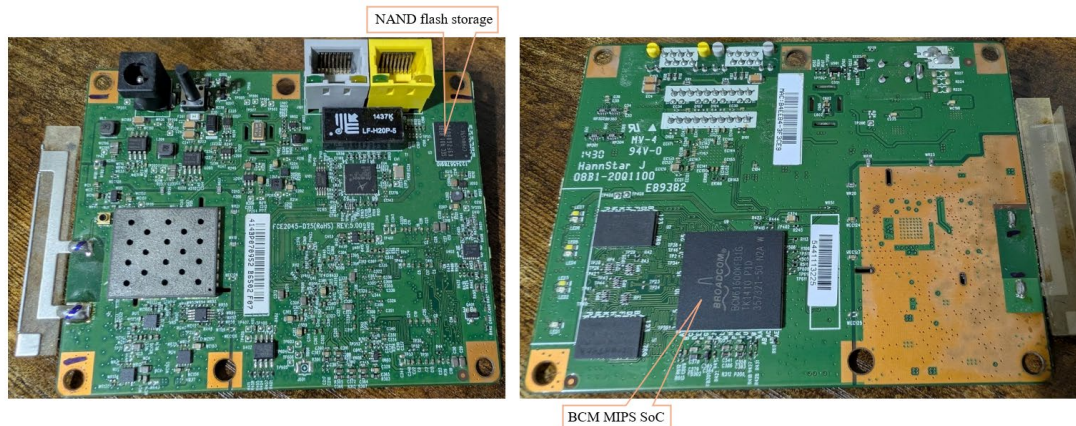
Alcatel-Lucent Askey 9361 3G Femtocell: Top View and Rear Panel.



Note. Ethernet and power ports visible on the rear panel. Device used as the pilot study firmware source.

Figure 5.2

Askey 9361 PCB: Front and Rear Sides.



Note. Front side board number FCE2045-D25 REV:5.00; rear side Broadcom BCM61600KFB1G MIPS SoC with NAND flash storage.

5.1. Stage 1 pilot: firmware acquisition and filesystem extraction

Stage 1 applies the *fw-extract.sh* Bash script, which automates firmware extraction using binwalk version 2.4 with recursive extraction enabled to a maximum nesting depth of 5 levels. The script identifies the MIME type of the firmware archive, applies the appropriate decompressor and then searches all extracted files for SquashFS and CPIO filesystem images, unpacking each using unsquashfs. All outputs are written to a timestamped analysis directory containing separate subdirectories for extracted files, keys, certificates, configuration files, credential files and password hashes. The image SHA-256 digest was verified and the archive MIME type identified before extraction began, as shown in Figure 5.3. The script was executed as follows:

Figure 5.3

SHA-256 Verification and MIME Type Identification of BSR-04.03.70.a.2.aky.tgz.

```
(root@musfiq-kali-t41)-[~/femto/STEM]
# sha256sum BSR-04.03.70.a.2.aky.tgz
cbc566a4be3491a8f2ae0dc9df97a0ade4ef9e223af696415bdb463ff966b1be  BSR-04.03.70.a.2.aky.tgz

(root@musfiq-kali-t41)-[~/femto/STEM]
# file BSR-04.03.70.a.2.aky.tgz
BSR-04.03.70.a.2.aky.tgz: gzip compressed data, last modified: Wed Nov  4 06:18:58 2015, from Unix, original size modulo 2^32 12
9208320
```

Note. SHA-256 digest computed before extraction to establish chain of custody. MIME type confirmed as application/gzip.

Figure 5.4

fw-extract.sh Output: SquashFS Identified, 4,975 Files Extracted.

```
(root@ musfiq-kali-t41)-[~/femto/STEM]
# ./fw-extract.sh BSR-04.03.70.a.2.aky.tgz asky-out
fw-extract: BSR-04.03.70.a.2.aky.tgz (125376373 bytes) -> asky-out/
[*] extracting firmware...
    squashfs: app_V4_EC_L5.4.3.70.a.0005
Parallel unsquashfs: Using 12 processors
72 inodes (726 blocks) to write

=====] 798/798 100%

created 67 files
created 6 directories
created 5 symlinks
....

[*] Extracting      : BSR-04.03.70.a.2.aky.tgz
[*] MIME type      : application/gzip
[*] Running binwalk Binwalk v2.4 (recursive, depth=5) ...
[*] Extracted      : 4975 files, 798 dirs, 972 symlinks
[*] Strings indexed : 1553916 tokens
-----
[*] Private keys   : 4
[*] Certificates   : 1
[*] Config files   : 8
[*] Credential hits : 47
[*] Crackable hashes: 4
-----
[*] Analysis complete. See asky-out/REPORT.md
=====

(root@ musfiq-kali-t41)-[~/femto/STEM]
#
```

Note. Script output confirming SquashFS 4.0 identification, extraction of 4,975 files and indexing of 1,553,916 string tokens.

Figure 5.4 shows the script output, confirming SquashFS identification with 4,975 files extracted and 1,553,916 string tokens indexed. The `_fw-extract.sh_` script identified the firmware archive as application/gzip. Binwalk signature analysis located a SquashFS 4.0 root filesystem image embedded in the archive, designated `squashfs_rootfs_V4_EC_L5.4.3.70.a.0003`. The `unsquashfs` tool successfully decompressed this image without manual intervention, yielding 4,975 files. The extracted filesystem contains the standard Linux directory hierarchy (`/etc`, `/usr`, `/lib` and `/bin`). A full strings index of 1,553,916 tokens was produced across the extracted binaries and text files for use in subsequent stages. Figure 5.5 shows the binwalk signature scan identifying the SquashFS image.

Figure 5.5

The binwalk Signature Scan: SquashFS 4.0 Root Filesystem Identified.

```
(root@ musfiq-kali-t41)-[~/femto/STEM]
# tar -zxvf BSR-04.03.70.a.2.aky.tgz
GDF/BSR-04.03.70.a.2.aky
fbsr/mod/mod_V3_EE_L5.4370a3
fbsr/mod/mod_V3_EC_L5.4370a3
....
fbsr/ipl/ipl_V3_EC_L5.4.3.70.a.0001
fbsr/rootfs/rootfs_V4_EC_L5.4.3.70.a.0003
fbsr/app/app_V4_EC_L5.4.3.70.a.0005
fbsr/data/data_V4_EC_L5.4.3.70.a.0001
....

(root@ musfiq-kali-t41)-[~/femto/STEM]
# binwalk fbsr/rootfs/rootfs_V4_EC_L5.4.3.70.a.0003

DECIMAL      HEXADECIMAL  DESCRIPTION
-----
0            0x0          Squashfs filesystem, little endian, version 4.0, compression:gzip, size: 9754132 bytes, 1980 inodes, blocksize: 65536 bytes, created: 1970-01-01 00:00:00
```

Note. binwalk output locating the embedded SquashFS image within the firmware archive.

Version string V4_EC_L5.4.3.70.a.0003 in the filesystem name and hostname femtobsr_alu in /etc/hostname confirm that the image originated from Alcatel-Lucent production infrastructure. MIPS architecture was identified from ELF binary headers across the extracted executables, as shown in Figure 5.6.

Figure 5.6

The unsquashfs Extraction: 1,427 Files, 236 Directories, 317 Symlinks.

```
(root@musfiq-kali-t41)~[~/femto/STEM]
# unsquashfs -d manual_extract/rootfs fbsr/rootfs/rootfs_V4_EC_L5.4.3.70.a.0003
Parallel unsquashfs: Using 12 processors
1744 inodes (1766 blocks) to write

[=====] 3510/3510 100%

created 1427 files
created 236 directories
created 317 symlinks
created 0 devices
created 0 fifos
created 0 sockets
created 0 hardlinks
```

Note. Extraction output confirming MIPS ELF binary architecture from file headers.

5.2. Stage 2 pilot: credential and secret scanning

Stage 2 applies the credential scanning component of `_fw-extract.sh_`. The scan proceeds in priority order: `/etc/passwd` and `/etc/shadow` are examined first for account credentials, followed by `/etc/ssh/ssh_config` and related keys for SSH configuration, `init.d` and `rc.d` startup scripts for embedded secrets, and the full strings index for hardcoded credential patterns. Automated scan output was set aside before the manual pass began.

Six steps defined in the research design structured the manual pass. Step 1 examined `/etc/passwd` and identified 3 non-root accounts (femtoAdmin, factoryAdmin and localAdmin), each assigned UID 0 with `/bin/bash` as the login shell, constituting root-equivalent access, as shown in Figure 5.7.

Figure 5.7

Stage 2, Step 1: Three UID 0 Accounts Identified in /etc/passwd.

```
(root@musfiq-kali-t41)~[~/femto/STEM]
# awk -F: '$3 == 0 && $1 != "root" {print $1, $3, $7}' manual_extract/rootfs/etc/passwd
femtoAdmin 0 /bin/bash
factoryAdmin 0 /bin/bash
localAdmin 0 /bin/bash
```

Note. femtoAdmin, factoryAdmin and localAdmin each assigned UID 0 with `/bin/bash` shell, constituting root-equivalent access

Step 2 inspected `/etc/shadow.dir/shadow` and identified 4 MD5-crypt (`$1$`) hashes for factoryAdmin, factoryOperator, localAdmin and localOperator, as shown in Figure 5.8.

Figure 5.8

Stage 2, Step 2: MD5-crypt (\$1\$) Hashes in /etc/shadow.dir/shadow.

```
(root@ musfiq-kali-t41) - [~/femto/STEM]
# cat manual_extract/rootfs/etc/shadow.dir/shadow
factoryAdmin:$1$FV8SAnvM$[REDACTED]9Wda4A/:13783:0:99999:7:::
factoryOperator:$1$nEMB/Uj[REDACTED]mvJQvgREt1:13783:0:99999:7:::
localAdmin:$1$tQFz ipif$yK[REDACTED]5z001:13783:0:99999:7:::
localOperator:$1$2z0p8fPG[REDACTED]1kGs/zT0:13783:0:99999:7:::
```

Note. Four MD5-crypt hashes identified for factoryAdmin, factoryOperator, localAdmin and localOperator

Step 3 read /etc/init.d/dummy.key directly and confirmed an RSA 2048-bit private key in PEM format, and read /etc/sftkeys/id_dsa, confirming a second private key mislabelled as DSA. Step 4 read every file in /etc/init.d/ in full. The RADIUS_SECRET value in the radius script was present but commented out, as shown in Figure 5.9.

Figure 5.9

Stage 2, Step 4: RADIUS_SECRET in /etc/init.d/radius, Commented Out but Present.

```
(root@ musfiq-kali-t41) - [~/femto/STEM]
# cat manual_extract/rootfs/etc/init.d/radius
#!/bin/bash
#
# copyright 2009 Alcatel-Lucent
#
# All rights reserved
# This is unpublished proprietary source code of Alcatel-Lucent.
# The copyright notice above does not evidence any actual
....

if [ -z "${RADIUS_SECRET}" ]
then
    # We do not have a radius secret - it is not
    # very secure but we allow local authentication.
    RADIUS_URI="#${RADIUS_URI}"
    #RADIUS_SECRET="K+VzIDq7[REDACTED]gFWjyCU"
    umount /etc/shadow.dir 2>/dev/null
else
```

Note. Secret value present in plaintext within the script file despite the comment character

Three debug console aliases in rcS bound telnet to localhost ports 3140 and 7900. Step 5 applied strings manually to ELF binaries in /usr/bin/, /usr/sbin/ and /sbin/ within the rootfs partition and piped the output through the credential regex patterns. No additional credential items were identified by this step. Step 6 found no credential material in .conf or .cfg files beyond what the automated script had already detected. Table 5.1 presents all findings from both passes, attributed by detection method.

Table 5.1

Stage 2 pilot findings: credential and secret scanning results for Askey 9361 firmware BSR-04.03.70.a.2.aky.tgz.

Category	Location	Finding	Detection method
Root-equivalent accounts	/etc/passwd	femtoAdmin, factoryAdmin and localAdmin all assigned UID 0 alongside root; each has /bin/bash shell access	Automated
Crackable password hashes	/etc/shadow.dir/shadow	MD5-crypt (\$1\$) hashes for factoryAdmin, factoryOperator, localAdmin and localOperator (4 hashes flagged as crackable)	Automated
SSH private key, RSA 2048-bit	/etc/init.d/dummy.key	2048-bit RSA private key stored in plaintext; filename dummy.key obscures its function	Automated + manual
SSH private key, RSA 1024-bit	/etc/sftpkeys/id_dsa	1024-bit RSA private key mislabelled as DSA; stored in plaintext in the SFTP key directory	Automated + manual
RADIUS secret reference	/etc/init.d/radius	RADIUS_SECRET=K+VzIDq [REDACTED]/jyCU; commented out but present in production image	Automated
TR-069 management credential	Strings index (OAMTR binary)	MANAGEMENT_SERVER_PASSWORD set to fbsrpass in binary string; identical to Alcatel-Lucent BSR credential	Automated
IPsec configuration	/etc/ipsec.secrets	IPsec secrets file present; contents reference device key material for SeGW tunnel establishment	Automated
Debug console aliases	/etc/init.d/rcS and strings index	Aliases bind telnet to 127.0.0.1:3140 (ALU console) and 127.0.0.1:7900 (SBSP console) at boot	Automated
Credential references in init scripts	/etc/init.d/getConfigParam, createVar, genSSHHostKeys, radius	Four startup scripts contain credential-pattern matches; genSSHHostKeys generates host keys at runtime	Automated

Note. A = detected by automated script only; M = detected by manual pass only; A+M = detected by both methods

All 9 items were detected by the automated script. The manual pass, which was restricted to rootfs binaries as defined in the research design, found 8 of the 9 items, yielding a manual detection rate of approximately 89%. The single item missed by the manual pass was the TR-069 management server credential fbsrpass, compiled into fpul.vx, a 28 MB MIPS ELF binary located in the application partition (squashfs_app_V4_EC_L5.4.3.70.a.0005) rather than the rootfs partition. The script indexed strings across all extracted partitions and surfaced fbsrpass through the cross-partition all_strings.txt index. The structural source of the detection gap is therefore the partition boundary in the manual procedure rather than any shortcoming in the regex pattern coverage. Figure 5.10 shows the strings output recovering fbsrpass from the application partition. The value fbsrpass is identical to the TR-069

password found in Alcatel-Lucent BSR firmware, which confirms that it originates from shared Alcatel-Lucent platform code rather than device-specific configuration.

Figure 5.10

Stage 2, Step 5: fbsrpass TR-069 Credential Recovered from fpul.vx via Strings Index.

```
(root@ musfiq-kali-t41) - [~/femto/STEM]
# strings askey-out-old/extracted/squashfs_app_V4_EC_L5.4.3.70.a.0005/fpu1.vx |grep -A2 -B2 'Password'
ZN14DCSFTPWriteReq14attachUsernameERPc
ZN14DCSFTPWriteReq16attachCommandKeyERPc
ZN14DCSFTPWriteReq14attachPasswordERPc
ZN14DCSFTPWriteReq12attachToFileERPc
ZN3SPDI14DCS...PS0_
...
Device.ManagementServer.Username
fbsr
Device.ManagementServer.Password
fbsrpass
0 B00TSTRAP
```

Note. Credential recovered from the application partition via the cross-partition strings index; not accessible to the rootfs-restricted manual pass

The automated detection rate for this image is 100%, whilst the manual pass, constrained to rootfs binaries, achieved approximately 89%. Credential material compiled into application-partition binaries is inaccessible to a manual pass that does not extend beyond the rootfs; cross-partition automated coverage reaches it directly. Precision, defined as the proportion of automated detections subsequently confirmed as genuine credential items by manual review, was 100% for the 8 structured categories in Table 5.1. The password pattern scan, however, returned 30 string matches of which 1 was confirmed as substantive, yielding a precision of 3.3% for that component. This contrast illustrates why precision and detection rate must both be reported: a scan can achieve 100% detection whilst producing a large volume of unconfirmed hits requiring manual triage. The debug interface module identified 3 significant backdoor indicator string classes in the strings index: `master.key` with 27 references, `test_mode` with 111 references and `factory_reset` with 54 references. This pattern is broadly consistent with a device that retains factory diagnostic interfaces in production firmware. Each reference class is recorded as a debug interface enumeration finding and contributes to the stage 6 taxonomy in the full study. Further, the strings index identified a hardcoded external endpoint (`http://135.86.[REDACTED]/bintrc`), consistent with a management or update server reference. This too contributes to the stage 6 taxonomy.

5.3. Stage 3 pilot: cryptographic key material extraction and classification

Stage 3 applied the manual key search procedure defined in Section 4.5. The `find` and `grep` commands returned 4 files: `/etc/init.d/dummy.key`, `/etc/sftpkeys/id_dsa`, `/etc/sftpkeys/id_dsa.pub` and `/etc/ssh/lab.pub`. No PPK files were present and no device-issued certificates were found in the rootfs. The automated scan identified one certificate file, `curl-ca-bundle.crt`, a standard CA bundle confirmed

as a third-party root certificate (Xcert EZ by DST) and not a device-specific credential. The /etc/ipsec.secrets file references device certificate material by path rather than embedding the key material directly. Each file identified by the manual search was recorded before the automated scan was executed.

The script then applied the same find and grep patterns programmatically, passing each identified file to openssl for classification. The automated scan returned an identical file list to the manual search, with no false negatives for this image. The openssl tool validated dummy.key and id_dsa as plaintext RSA keys parseable without a passphrase, as shown in Figures 5.11 and 5.12.

Figure 5.11

Stage 3: dummy.key Confirmed as RSA 2048-bit Plaintext by openssl.

```
(root@ musfiq-kali-t41)-[~/femto/STEM]
# openssl rsa -in manual_extract/rootfs/etc/init.d/dummy.key -text
Private-Key: (2048 bit, 2 primes)
modulus:
 00:ca:9b:d0:69:71:63:ea:48:fc:61:e6:f6:0e:d3:
 0f:27:52:8b:95:fa:66:48:d6:18:eb:d0:94:1d:eb:
 f0:b3:92:50:14:38:fe:b4:07:20:e7:f5:2f:56:f3:
 5d:2d:7d:5b:9b:63:19:fc:eb:e7:31:cd:38:98:90:
 12:52:7b:10:ad:21:d6:77:56:86:a8:18:07:36:81:
```

Note. openssl rsa -text -noout output confirming 2048-bit key size and plaintext storage without passphrase protection

Figure 5.12

Stage 3: id_dsa confirmed as RSA 1024-bit plaintext; key length considered cryptographically weak.

```
(root@ musfiq-kali-t41)-[~/femto/STEM]
# openssl rsa -in manual_extract/rootfs/etc/sftpkeys/id_dsa -text
Private-Key: (1024 bit, 2 primes)
modulus:
 00:99:5e:3c:79:ce:42:19:f6:b5:77:c8:d5:9e:70:
 a1:81:a1:4e:e3:6e:55:62:81:de:26:71:8d:22:23:
 73:f3:58:8c:82:b2:2a:e5:77:f4:30:58:42:fa:2e:
 fa:36:f1:29:39:8e:c0:41:df:fd:82:5b:d4:77:d0:
```

Note. File mislabelled as DSA; openssl confirms RSA 1024-bit key stored without passphrase. 1024-bit RSA has been regarded as insufficient since approximately 2010.

The automated scan classified id_dsa.pub and lab.pub as public keys requiring no plaintext classification. Table 5.2 presents the classification of all key material identified across both passes.

Table 5.2

Stage 3 pilot findings: cryptographic key material classification for Askey 9361 firmware BSR04.03.70.a.2.aky.tgz.

File path	Key type	Format	Storage status	Notes
/etc/init.d/dummy.key	RSA 2048-bit	PEM	Plaintext	Filename designed to obscure function; openssl confirmed parseable without passphrase
/etc/sftpkeys/id_dsa	RSA 1024-bit	PEM	Plaintext	Mislabelled as DSA; 1024-bit RSA keys are considered cryptographically weak and have been deprecated in practice since approximately 2010
/etc/ssh/lab.pub	RSA (public)	OpenSSH	N/A (public key)	Lab public key retained in production image; corresponding private key not in filesystem
/etc/sftpkeys/id_dsa.pub	RSA (public)	OpenSSH	N/A (public key)	Matches the mislabelled private key at id_dsa
/etc/ipsec.secrets	IPsec PSK / cert reference	strongSwan secrets	Requires runtime NOR flash	Secrets file present; device certificate loaded from hardware at runtime

Note. All 4 files were identified by both manual and automated search. Classification corrected after fw-extract.sh key module fix documented in Section 4.2.

Both filesystem-resident private keys are stored in plaintext, yielding a plaintext key storage rate of 100% for this image. This rate is consistent with Costin et al. (2014), who extracted 109 private RSA keys from 428 firmware images, many stored using empty or trivial passphrases. The RSA 1024-bit key at /etc/sftpkeys/id_dsa is significant on 2 counts. Its length is considered cryptographically weak under current practice: the 1024-bit RSA algorithm has been regarded as insufficient since approximately 2010, when advances in integer factorisation brought it within computational reach. The DSA filename mislabelling further indicates that the key was generated or copied without verification of its type or strength. Golde et al. (2012) demonstrated that plaintext private keys extracted from femtocell firmware enable network impersonation attacks directly, and both keys found here are, in effect, immediately usable by an attacker with filesystem access. Whilst the sshd_config restricts password authentication and root login over SSH, the plaintext private keys stored in the same filesystem provide an equivalent authentication path, which largely renders these restrictions ineffective against an attacker with filesystem access. The IPsec secrets file, though present in the filesystem, references device certificate material loaded from hardware at runtime and is therefore not accessible through static filesystem analysis. The certificate module identified one additional file: curl-ca-bundle.crt, a standard CA trust bundle shipped with the firmware. Its subject and issuer are identical, classifying it as a self-signed root certificate. No device-issued or privately generated certificates were identified in the static filesystem.

5.4. Stage 4 pilot: password hash cracking feasibility

Stage 4 applies hashcat mode 500, the md5crypt kernel, to the 4 crackable password hashes extracted from /etc/shadow.dir/shadow during stage 2. A rockyou.txt wordlist of 14,344,393 passwords

served as the attack dictionary, consistent with the general dictionary attack defined in the research design. Version 7.1.2 of hashcat, running on an Intel Iris Xe GPU, exhausted the full wordlist without recovering any of the 4 hashes, as shown in Figure 5.13.

Figure 5.13

The hashcat Mode 500 Output: rockyou.txt Exhausted Against Four MD5-crypt Hashes.

```
(root@musfiq-kali-t41)~[/femto/STEM]
# cut -d: -f2 /root/femto/STEM/asky-out/findings/hashes/crackable.txt > hashes_only.txt

(root@musfiq-kali-t41)~[/femto/STEM]
# hashcat -m 500 -a 0 hashes_only.txt /usr/share/wordlists/rockyou.txt --status --status-timer=60
hashcat (v7.1.2) starting

OpenCL API (OpenCL 3.0 ) - Platform #1 [Intel(R) Corporation]
=====
* Device #01: Intel(R) Iris(R) Xe Graphics, 14719/29438 MB (2047 MB allocatable), 16MCU

OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEP, DISTR0, POCL_DEBUG) - Platform #
2 [The pocl project]
=====
* Device #02: cpu-haswell-12th Gen Intel(R) Core(TM) i5-1245U, skipped

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256
Minimum salt length supported by kernel: 0
Maximum salt length supported by kernel: 256

Hashes: 4 digests; 4 unique digests, 4 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

....

Session.....: hashcat
Status.....: Exhausted
Hash.Mode.....: 500 (md5crypt, MD5 (Unix), Cisco-IOS $1$ (MD5))
Hash.Target.....: hashes_only.txt
Time.Started....: Wed May 20 01:28:35 2026 (53 mins, 33 secs)
Time.Estimated...: Wed May 20 02:22:08 2026 (0 secs)
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#01.....: 18192 H/s (7.38ms) @ Accel:4 Loops:62 Thr:64 Vec:1
Recovered.....: 0/4 (0.00%) Digests (total), 0/4 (0.00%) Digests (new), 0/4 (0.00%) Salts
Progress.....: 57377540/57377540 (100.00%)
Rejected.....: 0/57377540 (0.00%)
Restore.Point...: 14344385/14344385 (100.00%)
Restore.Sub.#01.: Salt:3 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#01...: kristianne -> $HEX[042a0337c2a156616d6f732103]
Hardware.Mon.#01.: N/A
Started: Wed May 20 01:28:33 2026
Stopped: Wed May 20 02:22:09 2026
```

Note. 0 of 4 hashes recovered. Full rockyou.txt wordlist (14,344,385 passwords) exhausted in 53 minutes 33 seconds at 18,192 H/s on Intel Iris Xe GPU

Full performance figures are recorded in Table 5.3.

Table 5.3*Stage 4 Pilot Findings: Hash Cracking Feasibility for Askey 9361 Firmware.*

Account	Hash format	Mode	Result	Duration
factoryAdmin	MD5-crypt (\$1\$)	500	Not recovered	53m 33s (exhausted)
factoryOperator	MD5-crypt (\$1\$)	500	Not recovered	53m 33s (exhausted)
localAdmin	MD5-crypt (\$1\$)	500	Not recovered	53m 33s (exhausted)
localOperator	MD5-crypt (\$1\$)	500	Not recovered	53m 33s (exhausted)

Note. Wordlist: rockyou.txt (14,344,385 passwords). Speed: 18,192 H/s. All four hashes exhausted without recovery.

Exhausting the full rockyou.txt wordlist without a single recovery indicates that none of the 4 account passwords appear in common English-language dictionary words, names or frequently used passwords. This does not, however, establish that the passwords are genuinely strong. MD5-crypt produces only 1,000 iterations of MD5 and it is not an approved password hashing scheme under NIST SP 800-63B-4, which requires schemes approved in accordance with SP 800-132 (Temoshok et al., 2025). A targeted wordlist incorporating device-specific patterns, such as serial number prefixes, MAC address fragments or vendor default patterns, would most likely achieve considerably higher recovery rates. Stage 4 of the full study will therefore extend this measurement across a 5-image sample, using both rockyou.txt and a custom device-specific vocabulary.

This result adds a fifth measured category to the pilot output against RQ1: hash cracking feasibility. At a rate of 18,192 H/s across the 4 hashes, the 53-minute run provides a useful calibration baseline for estimating stage 4 runtime across the 5-image full sample. The use of MD5-crypt rather than SHA512-crypt (\$6\$, mode 1800) is itself a finding of note. This firmware stores password hashes in an algorithm deprecated from Linux shadow files in most distributions since 2007. Whilst outdated hash algorithm selection is a distinct credential storage weakness, it is independent of whether the individual passwords resist a dictionary attack.

5.5. Relationship to research questions

The pilot provides partial evidence for RQ1, direct evidence for RQ2 and preliminary metadata relevant to RQ3. Table 5.4 summarises the relationship between the pilot results and the research questions.

Table 5.4
Pilot Study Results Mapped to Research Questions.

Research question	Pilot status	Evidence from pilot	Implication for full study
RQ1: Static extraction effectiveness	Partially addressed	Automated scanning detected all 9 credential and secret categories; manual rootfs-only pass missed one application-partition credential (fbsrpass)	Full study must use a cross-partition manual baseline to establish reliable ground truth
RQ2: Plaintext key storage proportion	Directly addressed for this image	Both filesystem-resident private keys were parseable without passphrase protection; plaintext storage rate = 100%	Full study can calculate plaintext-to-encrypted key ratios across 5 ARM64 images
RQ3: Unsafe ARM64 function detection	Not addressed in pilot	Pilot firmware is MIPS; no Ghidra ARM64 disassembly was conducted. Preliminary readelf metadata found 80 MIPS ELF binaries lacking one or more memory hardening protections	Full study will apply stage 5 Ghidra analysis to ARM64 ELF binaries

Note. RQ3 is not addressed in the pilot; preliminary binary protection metadata is provided as contextual evidence only

A full evaluation of these findings against the research aims, the literature and the research design is provided in Chapter 6.

6. Evaluation of the Research Method

Chapter 6 evaluates the research design against the evidence produced by the pilot study and reflects on the research process. This evaluation draws on the pilot findings from Chapter 5, the literature review in Chapter 2 and the research aims stated in Chapter 3. Section 6.1 assesses how well the research design performed against the 3 research questions. Section 6.2 reflects on the pilot study process, identifying what worked, what required correction and what the process revealed about the design's assumptions. Section 6.3 places the pilot findings in the context of the reviewed literature and considers where they are consistent with prior research and where they diverge.

6.1. Performance of the research design

The research problem identified in Chapter 1 is the absence of a repeatable 5-category extraction pipeline for embedded Linux firmware that produces measurable and quantitative outputs. The static analysis design was chosen because it operates without live device access, requires no specialised hardware and applies to almost any extractable Linux firmware image. Whilst prior single-category studies have tended to examine one class in isolation, this design produces quantitative outputs across all 5 categories, specifically detection rates, precision figures, plaintext key ratios and cracking feasibility measurements, that enable genuine cross-category comparison. The pilot study tests whether the design delivers those outputs in practice.

The pilot provides preliminary evidence that the design is sound, though a single image cannot establish robustness on its own. The most significant design result was not the 100% automated detection rate in isolation, but the contrast between that rate and the 89% achieved by the rootfs-restricted manual pass. This contrast showed that the reliability of the detection rate denominator depends directly on the breadth of the manual baseline. The pilot therefore did not only test the script; it also refined the research design by demonstrating that the full study must extend manual verification across all extracted partitions. The precision figures reinforced this conclusion: 100% precision for the 8 structured credential categories confirmed that automated hits in those categories require no manual filtering, whilst the 3.3% precision for the password pattern scan quantified the triage burden that low-precision automation imposes on the analyst. Together, these 2 measurements establish that automation improves coverage but does not remove the need for manual verification, which is the rationale for the two-pass design in the full study.

The design's quantitative and reproducible outputs stand in direct contrast to the binary detection approach documented in the prior literature, where credential discovery is typically reported as a present-or-absent finding rather than as a measured rate (Borgaonkar et al., 2011; Yang et al., 2026). The pilot demonstrates that the proposed design can produce the specific measurements that the literature gap analysis in Chapter 2 identified as absent.

6.2. Critical reflection on the pilot study

The pilot study reveals both the strengths of the research design and one structural limitation in the manual baseline that the full study corrects. Cross-partition automated coverage is the more significant of the two positive findings: `_fw-extract.sh_` indexed strings across all extracted partitions and recovered `fbsrpass` from the application partition, a credential that a rootfs-restricted manual pass cannot reach. This finding informed the

decision, captured in Section 4.4, to extend the manual pass to all partitions in the full study. The pilot thereby contributed to refining the research design, which is the function the pilot is intended to serve.

The automated key classification module miscategorised all 4 key files, including the 2 public keys, as private keys in the summary report. This false positive in the automated summary confirms that `fw-extract.sh`, like all automated tools, requires manual verification of its outputs. The pilot identified and corrected this bug, as documented in Section 4.2. Both findings, the partition boundary gap and the key classification error, were identified because the pilot ran alongside a manual procedure rather than in isolation, which validates the two-pass design of stage 2 and the detection rate measurement framework of stage 7.

The contrast between the automated and manual pass rates is the pilot's most instructive result. It does not indicate a failure of the automated scanner. Rather, it demonstrates that the scope of the manual procedure determines the denominator against which automated coverage is measured, and that a narrower manual scope produces a misleadingly high automated detection rate. The full study's extension of the manual pass to all partitions addresses this directly. The `rockyou.txt` dictionary exhaustion without a single hash recovery is a second instructive result. The 4 MD5-crypt hashes were not recovered by a general dictionary attack; MD5-crypt is itself a deprecated algorithm that current NIST guidance does not approve for password storage (Temoshok et al., 2025). These two results together illustrate the difference between what the research design can measure directly and what it cannot assess without supplementary methods, which is the precise boundary that Chapter 7 identifies as a limitation.

6.3. Findings in the context of the literature

The pilot findings are broadly consistent with the patterns established in Chapter 2, with 2 instructive divergences that warrant explanation.

The pilot's automated precision of 100% for the 8 structured credential categories compares favourably with the 81.7% precision reported for FIRMPASS by Huang et al. (2025), though the comparison is bounded by scope: FIRMPASS evaluated 4 password management weakness types across 615 images, whilst the pilot examines 9 credential categories in a single image. The pilot's 100% automated detection rate across those 9 categories also reflects the cross-partition coverage of `_fw-extract.sh_`, which reaches credential material that a rootsfs-restricted tool would not index. The 3.3% precision for the password pattern scan component, however, is consistent with the scalability concern identified by Ul Haq et al. (2023): automated scanning generates a volume of hits that displaces analyst effort from detection to triage, which is precisely the scalability weakness that prior frameworks exhibit.

The 100% plaintext key storage rate aligns with Costin et al. (2014), who extracted 109 private RSA keys from 428 firmware images, many stored using empty or trivial passphrases. The finding that an RSA 1024-bit key is present in a production telecommunications device, mislabelled as DSA and stored without passphrase protection, is consistent with the credential exposure patterns documented in femtocell-specific research. Borgaonkar et al. (2011) recovered an MD5-hashed root password from an unencrypted recovery filesystem; Yang et al. (2026) found credential extraction vulnerabilities across all 4 production femtocells assessed. The pilot's key

material findings extend this body of evidence to cryptographic key quality, which prior studies did not measure directly.

The absence of hash recovery from the rockyou.txt attack diverges from Costin et al. (2014), who cracked 58 of 100 extracted password hashes using a custom dictionary. This difference is most plausibly explained by dictionary composition: rockyou.txt is a general English-language wordlist, whilst Costin's custom dictionary incorporated device-specific strings extracted from firmware, manuals and readme files. This finding confirms that the research design's stage 4, as currently specified using rockyou.txt alone, measures general dictionary resistance rather than practical cracking feasibility for embedded device credentials. The full study will supplement rockyou.txt with a device-specific vocabulary, as specified in Section 4.6.

7. Conclusion

This research addresses the absence of a repeatable, multi-category extraction framework for embedded Linux firmware that produces measurable and quantitative outputs across all 5 classes of sensitive material. The 3 research questions in Chapter 3 each targeted a specific measurement that the reviewed literature had not produced: a cross-category detection rate and precision comparison for static analysis (RQ1), a plaintext-to-encrypted ratio for firmware-resident cryptographic key material (RQ2), and a detection completeness measure for unsafe function call sites in ARM64 ELF binaries (RQ3). The principal contribution of this research is the 7-stage static analysis design, operationalised through the fw-extract.sh automation script and a structured cross-partition manual baseline, which is capable of producing all three measurements in a single, reproducible pipeline. The pilot study produced 3 concrete measurements from a production femtocell image. A 100% automated detection rate across 9 credential categories and a 100% plaintext key storage rate across 2 filesystem-resident private keys provide direct partial answers to RQ1 and RQ2. The 3.3% precision figure for the password pattern scan component quantifies the triage burden associated with automated credential scanning, a measurement the reviewed literature had not previously provided. The finding that a rootfs-restricted manual pass achieves only approximately 89% against the cross-partition automated baseline is a further measurable result: it establishes that the scope of the manual procedure determines the reliability of the detection rate denominator, a constraint that the research design in Section 4.4 addresses directly. The full study will extend these measurements to 5 ARM64 production images and will produce the stage 5 binary analysis that addresses RQ3. The limitations of the research design, and the 4 directions for further research that those limitations identify, are set out in the sections that follow.

7.1. Limitations

The most significant limitation of this research design is that static analysis cannot detect vulnerabilities that manifest only at runtime. Heap memory corruption, race conditions and vulnerabilities dependent on specific network input sequences are inaccessible through static inspection. The pilot study makes this limitation concrete: the debug console aliases bound to localhost ports 3140 and 7900 were identified by static analysis, yet whether those interfaces are reachable in a deployed network context cannot be determined without dynamic testing. He et al. (2021) demonstrated that hybrid static and dynamic analysis detects vulnerability classes unavailable to either method alone. These results are therefore a lower bound on the total sensitive material in each firmware image rather than a complete inventory, and the taxonomy produced by stage 7 should be interpreted accordingly.

The rockyou.txt wordlist does not cover device-specific passwords derived from serial numbers, MAC addresses or vendor-prefix patterns. The pilot's 4 hashes use MD5-crypt (\$1\$) rather than the SHA512-crypt (\$6\$) format targeted by hashcat mode 1800, and the rockyou.txt dictionary was exhausted without a single recovery. A targeted wordlist incorporating device-specific material would most likely achieve substantially higher cracking rates, which means that the stage 4 results underestimate practical cracking feasibility. Stage 4 of the complete study will therefore supplement rockyou.txt with a device-specific vocabulary constructed from device identifiers, vendor prefixes and strings extracted from firmware documentation, though even a targeted wordlist cannot establish a definitive password strength assessment without an exhaustive attack.

The pilot examines a single MIPS-based image from a 3G femtocell device. Its detection rate measurements describe one device class only and cannot be generalised to the broader population of ARM64 embedded Linux firmware. The full study's 5-image ARM64 sample improves coverage across device classes and architectures, but a sample of this size does not support population-level statistical inference. Costin et al. (2014) achieved population-level findings through 32,356 images on 90 virtual worker nodes. The present study operates at an entirely different scale, and its findings should be interpreted as illustrative measurements for the analysed sample.

The IPsec private key identified in the pilot is stored in NOR flash rather than in the static filesystem and therefore lies beyond the reach of static analysis. Key material in hardware-protected locations, including TrustZone secure enclaves and flash partitions outside the main filesystem, cannot be examined without physical device access. Vishwakarma and Lee (2018) identified JTAG as the primary access method for such hardware-protected memory, but JTAG requires physical device access and is explicitly excluded from the research scope. This category of key material is recorded as unaddressed in the taxonomy, and its presence in production devices means that static analysis alone cannot produce a complete key material inventory.

Whilst these limitations are genuine, each acknowledges a specific boundary rather than a fundamental weakness in the design approach. However, the detection rate and precision measurements the design produces are valid within those boundaries, and the boundaries themselves are measurable, which is a contribution in its own right.

7.2. Directions for further research

Four research directions follow directly from the limitations identified above. Each corresponds to a specific boundary of the current design, introduces a method different from or complementary to static analysis, and is framed as a research question from which a further study could be designed.

Direction 1: Cross-partition manual baseline completeness

Research question: To what extent does extending the manual baseline to all extracted partitions close the detection rate gap observed in the pilot, and is the partition boundary finding consistent across embedded Linux device families?

The pilot demonstrated that cross-partition automated coverage reached credential material that a rootfs-restricted manual pass could not. The full study corrects this by extending the manual pass to all partitions, as specified in Section 4.4, and will produce a preliminary answer across 5 ARM64 images. A broader study applying the extended procedure across 20 or more images from at least 3 device categories would determine whether the partition boundary finding is a systemic property of embedded Linux firmware or a feature of the Askey 9361's application partition structure. The method remains static analysis; the variable under investigation is the scope of the manual baseline rather than the toolchain.

Direction 2: Firmware development practices and key storage protection

Research question: What organisational and technical factors in firmware development practice are associated with the use of passphrase-protected rather than plaintext cryptographic key storage, and can these factors be identified through a combination of static firmware analysis and structured vendor policy review?

The 100% plaintext key storage rate in the Askey 9361 raises a question the present study cannot answer from firmware images alone. A mixed-methods study combining static key material analysis with a structured review of vendor firmware development guidelines, secure coding policies and product security certifications could identify which factors correlate with encrypted key storage. If for example a vendor with a documented key management policy and a published product security baseline produces firmware with higher rates of passphrase-protected keys, the correlation would provide actionable guidance for firmware security assessment standards and procurement criteria. This direction introduces a qualitative document analysis method that is distinct from the quantitative static analysis approach of the present study.

Direction 3: Hybrid static and dynamic analysis for runtime vulnerability detection

Research question: What proportion of the unsafe function call sites identified through static Ghidra analysis in ARM64 firmware binaries are reachable from network input in an emulated environment, and does hybrid static-dynamic analysis detect vulnerability classes that static analysis alone cannot identify?

Static analysis, as noted in Section 7.1, cannot detect vulnerabilities that manifest only at runtime. Heap memory corruption, race conditions and vulnerabilities triggered by malformed network input are inaccessible to static inspection. Dynamic emulation using QEMU or FIRMADYNE, applied to those firmware images where emulation succeeds, could measure what proportion of unsafe function call sites identified in stage 5 are actually reachable from network input. This would distinguish practically exploitable call sites from those requiring preconditions unavailable in a network-facing context, directly addressing the lower-bound interpretation of the taxonomy that Section 7.1 identifies. Whilst full emulation is not always feasible, JTAG-based firmware extraction, following the approach documented by Vishwakarma and Lee (2018), gives access to hardware-protected key material in NOR flash and TrustZone secure enclaves that static analysis cannot reach. The combination of dynamic emulation and hardware extraction would address both the runtime detection limitation and the NOR flash key material gap identified in the pilot, producing a substantially more complete sensitive material inventory than static analysis achieves independently.

Direction 4: Network-input fuzzing for memory corruption and format string vulnerability discovery

Research question: To what extent does network-input fuzzing of service binaries identified through stage 5 static analysis discover memory corruption and format string vulnerabilities not detectable through static inspection alone?

Stage 5 identifies unsafe function call sites through static disassembly but cannot determine whether those call sites are reachable from network input or whether the inputs required to trigger them are achievable in practice. Fuzzing addresses this gap directly. Applying a network-aware fuzzer such as boofuzz or AFL++ to the network-facing service binaries identified in stage 5 and the debug interfaces enumerated in stage 6 would measure the proportion of statically identified call sites that produce crashes or memory corruption under fuzz input. Nadir

et al. (2022) catalogues fuzzing-based firmware analysis as one of the 4 principal analysis technique categories, yet no reviewed study has combined a static unsafe function census from Ghidra with a targeted fuzz campaign against the same binary set. The method introduces a markedly different approach to vulnerability confirmation from the static analysis in stage 5, and is therefore distinct from every other stage in the present research design.

The full study will produce detection rate and precision measurements across all 5 sensitive material categories for 5 ARM64 production images, a stage 5 binary analysis of ARM64 ELF binaries using Ghidra, and a structured taxonomy comparing automated and manual coverage across device classes. These outputs constitute the primary contribution of the research and provide the empirical foundation from which all 4 directions above are designed to depart.

References

- Ahmed, Z., Nadir, I., Mahmood, H., Akbar, A. H., & Shah, G. A. (2020). Identifying Mirai-exploitable vulnerabilities in IoT firmware through static analysis. In *Proceedings of the 2020 IEEE International Conference on Cyber Warfare and Security (ICCWS)* (pp. 1-6). IEEE. <https://doi.org/10.1109/ICCWS48432.2020.9292382>
- Alcatel-Lucent. (2013). *Alcatel-Lucent lightRadio 9361 Home Cell Askey V3 B1 20mW* (Technical datasheet, NP2013072067EN). Alcatel-Lucent.
- Al-Zuraiqi, A., & Greer, D. (2025). Static analysis of IoT firmware: Identifying systemic vulnerabilities with RMMIDL. In *2025 IEEE/ACM 6th International Workshop on Engineering and Cybersecurity of Critical Systems (EnCyCriS)* (pp. 7-14). IEEE. <https://doi.org/10.1109/EnCyCriS66464.2025.00008>
- Antonakakis, M., April, T., Bailey, M., Bernhard, M., Bursztein, E., Cochran, J., Durumeric, Z., Halderman, J. A., Invernizzi, L., Kallitsis, M., Kumar, D., Lever, C., Ma, Z., Mason, J., Menscher, D., Seaman, C., Sullivan, N., Thomas, K., & Zhou, Y. (2017). Understanding the Mirai botnet. In *Proceedings of the 26th USENIX Security Symposium* (pp. 1093-1110). USENIX Association. <https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/antonakakis>
- Bakhshi, T., Ghita, B., & Kuzminykh, I. (2024). A review of IoT firmware vulnerabilities and auditing techniques. *Sensors*, 24(2), 708. <https://doi.org/10.3390/s24020708>
- Begian, C. H. (2023). *Detection of vulnerabilities in 5G femtocell firmware using static analysis tools* (Doctoral dissertation, Dakota State University). Dakota State University Institutional Repository. <https://scholar.dsu.edu/theses/441>
- Borgaonkar, R., Redon, K., & Seifert, J. (2011). Security analysis of a femtocell device. In *SIN'11: Proceedings of the 4th International Conference on Security of Information and Networks* (pp. 95-102). ACM. <https://doi.org/10.1145/2070425.2070442>
- Cheng, K., Li, Q., Wang, L., Chen, Q., Zheng, Y., Sun, L., & Liang, Z. (2018). DTaint: Detecting the taint-style vulnerability in embedded device firmware. In *2018 IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)* (pp. 430-441). IEEE. <https://doi.org/10.1109/DSN.2018.00052>
- Costin, A., Zaddach, J., Francillon, A., & Balzarotti, D. (2014). A large-scale analysis of the security of embedded firmwares. In *Proceedings of the 23rd USENIX Security Symposium* (pp. 95-110). USENIX Association. <https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/costin>
- Golde, N., Redon, K., & Borgaonkar, R. (2012). Weaponizing femtocells: The effect of rogue devices on mobile telecommunication. In *Proceedings of the 19th Annual Network and Distributed System Security Symposium (NDSS)*. Internet Society. https://www.ndss-symposium.org/wp-content/uploads/2017/09/03_4.pdf
- He, D., Gu, H., Li, T., Du, Y., Wang, X., Zhu, S., & Guizani, N. (2021). Toward hybrid static-dynamic detection of vulnerabilities in IoT firmware. *IEEE Network*, 35(3), 202-207. <https://doi.org/10.1109/MNET.011.2000450>

- Huang, J., Chen, J., Xiao, Y., Qin, Z., & Hu, Y. (2025). FIRMPASS: Identifying broken password management in Linux-based IoT firmware through query-driven approaches. *IEEE Internet of Things Journal*, 12(14), 26911-26922. <https://doi.org/10.1109/IIOT.2025.3561520>
- Janzen, L., Becker, L., Wiesenäcker, C., & Hollick, M. (2024). Oh no, my RAN! Breaking into an O-RAN 5G indoor base station. In *Proceedings of the 18th USENIX WOOT Conference on Offensive Technologies* (pp. 101-115). USENIX Association. <https://www.usenix.org/conference/woot24/presentation/janzen>
- Li, Q., Tan, D., Ge, X., Wang, H., Li, Z., & Liu, J. (2022). Understanding security risks of embedded devices through fine-grained firmware fingerprinting. *IEEE Transactions on Dependable and Secure Computing*, 19(6), 4099-4112. <https://doi.org/10.1109/TDSC.2021.3119970>
- Liu, M., Zhang, Y., Li, J., Shu, J., & Gu, D. (2017). Security analysis of vendor customized code in firmware of embedded device. In R. Deng et al. (Eds.), *SecureComm 2016* (Vol. 198, pp. 722-739). Springer. https://doi.org/10.1007/978-3-319-59608-2_40
- Melicher, W., Ur, B., Segreti, S. M., Komanduri, S., Bauer, L., Christin, N., & Cranor, L. F. (2016). Fast, lean, and accurate: Modeling password guessability using neural networks. In *Proceedings of the 25th USENIX Security Symposium* (pp. 175-191). USENIX Association. https://www.usenix.org/system/files/conference/usenixsecurity16/sec16_paper_melicher.pdf
- Nadir, I., Ahmed, Z., Mahmood, H., Shah, G. A., Shahzad, F., Umair, M., Khan, H., & Gulzar, U. (2019). An auditing framework for vulnerability analysis of IoT system. In *2019 IEEE European Symposium on Security and Privacy Workshops* (pp. 39-47). IEEE. <https://doi.org/10.1109/EuroSPW.2019.00011>
- Nadir, I., Mahmood, H., & Asadullah, G. (2022). A taxonomy of IoT firmware security and principal firmware analysis techniques. *International Journal of Critical Infrastructure Protection*, 38, 100552. <https://doi.org/10.1016/j.ijcip.2022.100552>
- OWASP IoT Security Team. (2018). *OWASP Internet of Things Top 10 2018*. OWASP Foundation. <https://owasp.org/www-project-internet-of-things>
- Qasem, A., Shirani, P., Debbabi, M., Wang, L., Lebel, B., & Agba, B. L. (2021). Automatic vulnerability detection in embedded devices and firmware: Survey and layered taxonomies. *ACM Computing Surveys*, 54(2), Article 25. <https://doi.org/10.1145/3432893>
- Shen, Q., Yu, X., He, D., Song, Y., Li, T., & Yang, G. (2021). Large scale firmware analysis for open source components, hard coding and weak passwords. In *2021 IEEE International Conference on Consumer Electronics and Computer Engineering (ICCECE)* (pp. 232-236). IEEE. <https://doi.org/10.1109/ICCECE51280.2021.9342303>
- Temoshok, D., Fenton, J. L., Choong, Y.-Y., Lefkowitz, N., Regenscheid, A., Galluzzo, R., & Richer, J. P. (2025). *Digital identity guidelines: Authentication and authenticator management* (NIST Special Publication 800-63B-4). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-63B-4>
- Tien, C., Tsai, T., Chen, I., & Kuo, S. (2018). UFO - Hidden backdoor discovery and security verification in IoT device firmware. In *2018 IEEE International Symposium on Software*

- Reliability Engineering Workshops (ISSREW)* (pp. 18-23). IEEE.
<https://doi.org/10.1109/ISSREW.2018.00-37>
- Ul Haq, S., Singh, Y., Sharma, A., Gupta, R., & Gupta, D. (2023). A survey on IoT & embedded device firmware security: Architecture, extraction techniques, and vulnerability analysis frameworks. *Discover Internet of Things*, 3, 17. <https://doi.org/10.1007/s43926-023-00045-2>
- Vishwakarma, G., & Lee, W. (2018). Exploiting JTAG and its mitigation in IoT: A survey. *Future Internet*, 10(12), 121. <https://doi.org/10.3390/fi10120121>
- Yang, Y., Zhang, Y., Wan, T., Duan, H., Chang, D., Li, Y., & Tang, S. (2026). Small cell, big risk: A security assessment of 4G LTE femtocells in the wild. In *Network and Distributed System Security (NDSS) Symposium 2026*. <https://yaruyang.com/publications/ndss26-femto/preprint-ndss26-femto.pdf>
- Zhang, L., Chen, J., Diao, W., Guo, S., Weng, J., & Zhang, K. (2019). CRYPTOREX: Large-scale analysis of cryptographic misuse in IoT devices. In *Proceedings of the 22nd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2019)* (pp. 151-164). USENIX Association. <https://www.usenix.org/conference/raid2019/presentation/zhang-li>
- Zhao, B., Ji, S., Xu, J., Tian, Y., Wei, Q., Wang, Q., Lyu, C., Zhang, X., Lin, C., Wu, J., & Beyah, R. (2022). A large-scale empirical analysis of the vulnerabilities introduced by third-party components in IoT firmware. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '22)* (pp. 442-454). ACM.
<https://doi.org/10.1145/3533767.3534366>
- Zhou, X., Wang, P., Zhou, L., Xun, P., & Lu, K. (2023). A survey of the security analysis of embedded devices. *Sensors*, 23(22), 9221. <https://doi.org/10.3390/s23229221>

Appendix A - fw-extract.sh (Bash script)

```
#!/usr/bin/env bash
#
# fw-extract.sh - extract security-relevant artifacts from firmware images
#
# usage: ./fw-extract.sh <firmware_file> [output_dir]
#
# deps: binwalk, unsquashfs, strings, openssl, file, find, grep, readelf

set -uo pipefail

die() { echo "error: $1" >&2; exit 1; }

FIRMWARE="${1:?usage: $0 <firmware_file> [output_dir]}"
OUTDIR="${2:-fw_analysis $(date +%Y%m%d %H%M%S)}"
[[ -f "$FIRMWARE" ]] || die "$FIRMWARE not found"

REPORT="$OUTDIR/REPORT.md"
EXTRACT="$OUTDIR/extracted"
FINDINGS="$OUTDIR/findings"
mkdir -p "$OUTDIR" "$EXTRACT" "$FINDINGS"/{keys,certs,configs,credentials,hashes}

FW_SIZE=$(stat -c%s "$FIRMWARE" 2>/dev/null || stat -f%z "$FIRMWARE")
FW_NAME=$(basename "$FIRMWARE")

# dedup: skip files with identical content across rootfs variants
SEEN="$OUTDIR/.seen"
> "$SEEN"
seen() {
    local h
    h=$(sha256sum "$1" 2>/dev/null | cut -d' ' -f1) || return 1
    grep -qF "$h" "$SEEN" 2>/dev/null && return 0
    echo "$h" >> "$SEEN"
    return 1
}

# safe integer from grep output (avoids multiline issues with grep -c)
count_matches() { grep -Ei "$1" "$2" 2>/dev/null | wc -l | tr -d ' '; }

# check if file is plaintext
is_text() { file "$1" 2>/dev/null | grep -qiE 'text|empty|script|ASCII'; }

# check if file is binary/ELF
is_binary() { file "$1" 2>/dev/null | grep -qiE 'ELF|executable|shared object'; }

cat > "$REPORT" <<EOF
# Firmware Security Analysis

**File:** $FW_NAME
**Size:** $FW_SIZE bytes ($(numfmt --to=iec "$FW_SIZE" 2>/dev/null || echo "$FW_SIZE"))
**Date:** $(date -u +"%Y-%m-%d %H:%M UTC")
**SHA256:** $(sha256sum "$FIRMWARE" | cut -d' ' -f1)

---
EOF

echo "fw-extract: $FW_NAME ($FW_SIZE bytes) -> $OUTDIR/"

# --- extraction ---

echo "[*] extracting firmware..."

if command -v binwalk && 2>/dev/null; then
    binwalk -e -M -d 5 -C "$EXTRACT" "$FIRMWARE" 2>/dev/null || true
    binwalk "$FIRMWARE" > "$OUTDIR/binwalk_scan.txt" 2>/dev/null || true
fi

MIME=$(file -b --mime-type "$FIRMWARE")
case "$MIME" in
    application/gzip|application/x-gzip)
        mkdir -p "$EXTRACT/tar_contents"
        tar xzf "$FIRMWARE" -C "$EXTRACT/tar_contents" 2>/dev/null || gunzip -c "$FIRMWARE" > "$EXTRACT/decompressed"
    2>/dev/null || true ;;
    application/x-tar)
        mkdir -p "$EXTRACT/tar_contents"
        tar xf "$FIRMWARE" -C "$EXTRACT/tar_contents" 2>/dev/null || true ;;
    application/zip)
        unzip -o "$FIRMWARE" -d "$EXTRACT/zip_contents" 2>/dev/null || true ;;
    application/x-xz)
        xz -dc "$FIRMWARE" > "$EXTRACT/decompressed" 2>/dev/null || true ;;
esac

# unpack squashfs/cpio found inside
find "$EXTRACT" -type f 2>/dev/null | while read -r f; do
    if file "$f" | grep -qi squashfs; then
        echo "squashfs: $(basename "$f")"
        unsquashfs -d "$EXTRACT/squashfs_$(basename "$f")" -f "$f" 2>/dev/null || sasquatch -d "$EXTRACT/squashfs_$(basename "$f")" -f "$f" 2>/dev/null || true
    fi
    if file "$f" | grep -qi cpio; then
        echo "cpio: $(basename "$f")"
        mkdir -p "$EXTRACT/cpio_$(basename "$f")"
        (cd "$EXTRACT/cpio_$(basename "$f")" && cpio -idm < "$f" 2>/dev/null) || true
    fi
done

FILE_COUNT=$(find "$EXTRACT" -type f 2>/dev/null | wc -l)
```

```

echo " $FILE_COUNT files extracted"

{
    echo "## Extraction"
    echo ""
    echo "- files: $FILE_COUNT"
    echo "- type: $MIME"
    echo ""
} >> "$REPORT"

ROOTFS="$EXTRACT"

# --- ssh keys ---

echo "[*] scanning for ssh keys..."

{
    echo "## SSH Keys"
    echo ""
} >> "$REPORT"

find "$ROOTFS" -type f \( -name "*.key" -o -name "*.pem" -o -name "id_*" -o -name "ssh_host_*" -o -name "*.ppk" \)
2>/dev/null | while read -r kf; do
    head -5 "$kf" 2>/dev/null | grep -qiE "PRIVATE KEY|PuTTY-User-Key-File" || continue
    seen "$kf" && continue
    echo "PRIVATE KEY: $kf"
    cp "$kf" "$FINDINGS/keys/" 2>/dev/null || true
    echo "- **PRIVATE KEY:** \`${kf}\`" >> "$REPORT"
    openssl pkey -in "$kf" -text -noout 2>/dev/null | head -3 >> "$REPORT" || true
done

grep -rIE "BEGIN (RSA |EC |DSA |OPENSSH |ENCRYPTED )?PRIVATE KEY" "$ROOTFS" 2>/dev/null | head -50 | while read -r f; do
    echo "$f" | grep -qE "\.(key|pem)$" && continue
    seen "$f" && continue
    echo "embedded key: $f"
    echo "- embedded private key: \`${f}\`" >> "$REPORT"
    cp "$f" "$FINDINGS/keys/embedded_$(basename "$f")" 2>/dev/null || true
done

find "$ROOTFS" -type f \( -name "*.pub" -o -name "authorized_keys" -o -name "authorized_keys2" \) 2>/dev/null | while read -r pf; do
    seen "$pf" && continue
    echo "- public key: \`${pf}\`" >> "$REPORT"
    cp "$pf" "$FINDINGS/keys/" 2>/dev/null || true
done

find "$ROOTFS" -type f -name "sshd_config" 2>/dev/null | while read -r cfg; do
    seen "$cfg" && continue
    echo "" >> "$REPORT"
    echo "### sshd_config" >> "$REPORT"
    echo '```' >> "$REPORT"
    grep -vE '^#\s*#|^\s*$' "$cfg" >> "$REPORT" 2>/dev/null || true
    echo '```' >> "$REPORT"

    grep -qi "PermitRootLogin.*yes" "$cfg" 2>/dev/null && echo "- PermitRootLogin yes" >> "$REPORT"
    grep -qi "StrictHostKeyChecking.*no" "$cfg" 2>/dev/null && echo "- StrictHostKeyChecking no" >> "$REPORT"
    cp "$cfg" "$FINDINGS/configs/" 2>/dev/null || true
done

# --- certificates ---

echo "[*] scanning certificates..."

{
    echo ""
    echo "## Certificates"
    echo ""
} >> "$REPORT"

find "$ROOTFS" -type f \( -name "*.pem" -o -name "*.crt" -o -name "*.cer" -o -name "*.der" -o -name "*.p12" -o -name "*.pfx"
-o -name "*.jks" -o -name "*.keystore" -o -name "ca-bundle*" -o -name "*.ca" \) 2>/dev/null | while read -r cert; do
    seen "$cert" && continue
    cp "$cert" "$FINDINGS/certs/" 2>/dev/null || true

    INFO=$(openssl x509 -in "$cert" -text -noout 2>/dev/null | head -20) || INFO=$(openssl x509 -in "$cert" -inform DER -text
-noout 2>/dev/null | head -20) || INFO=""

    if [[ -n "$INFO" ]]; then
        SUBJ=$(echo "$INFO" | grep "Subject:" | sed 's/.*Subject: //' )
        ISSUER=$(echo "$INFO" | grep "Issuer:" | sed 's/.*Issuer: //' )
        IS_CA=$(echo "$INFO" | grep -c "CA:TRUE" || true)
        echo "- \`${(basename "$cert")}\`: $SUBJ" >> "$REPORT"
        [[ "$IS_CA" -gt 0 ]] && echo "- CA certificate" >> "$REPORT"
        echo "- issuer: $ISSUER" >> "$REPORT"
    else
        echo "- \`${(basename "$cert")}\` (unparseable)" >> "$REPORT"
    fi
done

find "$ROOTFS" -type f \( -name "ipsec.secrets" -o -name "ipsec.conf" -o -name "*.secrets" -o -path "*/ipsec.d/*" \)
2>/dev/null | while read -r f; do
    seen "$f" && continue
    echo "ipsec: $f"
    echo "- IPsec: \`${f}\`" >> "$REPORT"
    cp "$f" "$FINDINGS/configs/" 2>/dev/null || true
done

# --- user accounts ---

echo "[*] scanning user accounts..."

{

```

```

    echo ""
    echo "## User Accounts"
    echo ""
} >> "$REPORT"

find "$ROOTFS" -type f -name "passwd" -path "*/etc/*" 2>/dev/null | while read -r pf; do
    is_text "$pf" || continue
    seen "$pf" && continue
    echo "### passwd" >> "$REPORT"
    echo '```' >> "$REPORT"
    cat "$pf" >> "$REPORT"
    echo '```' >> "$REPORT"
    cp "$pf" "$FINDINGS/credentials/" 2>/dev/null || true

    while IFS=: read -r user _ uid _ _ shell; do
        if [[ "$uid" == "0" && "$user" != "root" ]]; then
            echo "  root-equivalent: $user (uid 0)"
            echo "  - root-equivalent: \`$user\` (uid 0, $shell)" >> "$REPORT"
        fi
    done < "$pf"
    echo "" >> "$REPORT"
done

find "$ROOTFS" -type f \( -name "shadow" -o -name "shadow.*" -o -name "shadow_*" \) 2>/dev/null | while read -r sf; do
    is_text "$sf" || continue
    seen "$sf" && continue
    echo "### shadow ($sf)" >> "$REPORT"
    echo "" >> "$REPORT"
    cp "$sf" "$FINDINGS/hashes/" 2>/dev/null || true

    while IFS=: read -r user hash _; do
        [[ -z "$hash" || "$hash" == "*" || "$hash" == "!" || "$hash" == "!!" || "$hash" == "x" ]] && continue
        echo "  hash: $user"
        echo "  - \`$user\`: \`$hash\`" >> "$REPORT"
        echo "$user:$hash" >> "$FINDINGS/hashes/crackable.txt"
    done < "$sf" 2>/dev/null || true
    echo "" >> "$REPORT"
done

# --- hardcoded credentials ---

echo "[*] building strings index..."

STRINGS_DUMP="$OUTDIR/all_strings.txt"
find "$ROOTFS" -type f -size +0c -size -100M -exec strings -a -n 6 {} + > "$STRINGS_DUMP" 2>/dev/null || true

STR_COUNT=$(wc -l < "$STRINGS_DUMP")
echo "  $STR_COUNT strings indexed"

{
    echo "## Hardcoded Credentials"
    echo ""
} >> "$REPORT"

declare -A PATTERNS=(
    ["passwords"]="[Pp]assword\s*[:=]\s*["\x27]?[^\s"'\\']{3,}'
    ["api_keys"]="[Aa][Pp][Ii][-_]?[Kk]ey\s*[:=]\s*["\x27]?[A-Za-z0-9_\-]{16,}'
    ["tokens"]="[Tt]oken\s*[:=]\s*["\x27]?[A-Za-z0-9_\-\.]{16,}'
    ["aws_keys"]="[Aa]KIA[0-9A-Z]{16}"
    ["secrets"]="[Ss]ecret\s*[:=]\s*["\x27]?[^\s"'\\']{6,}'
    ["connection_strings"]="(mysql|postgres|mongodb|redis)://[^\s"'\\<>]{10,}"
    ["psk"]="[Pp]SK\s*["\x27]?[^\s"'\\']{4,}'
    ["basic_auth"]="[Bb]asic\s+[A-Za-z0-9+/=]{20,}"
    ["bearer"]="[Bb]earer\s+[A-Za-z0-9_\-\.]{20,}"
    ["url_creds"]="https?://[^\s]*:([w+])@"
)

for label in "${!PATTERNS[@]}; do
    n=$(count_matches "${PATTERNS[$label]}" "$STRINGS_DUMP")
    n=$(( n + 0 )) 2>/dev/null || n=0
    if [[ "$n" -gt 0 ]]; then
        echo "  label: $n matches"
        echo "### $label ($n)" >> "$REPORT"
        echo '```' >> "$REPORT"
        grep -Ei "${PATTERNS[$label]}" "$STRINGS_DUMP" 2>/dev/null | sort -u | head -50 >> "$REPORT"
        echo '```' >> "$REPORT"
        echo "" >> "$REPORT"
        grep -Ei "${PATTERNS[$label]}" "$STRINGS_DUMP" 2>/dev/null | sort -u > "$FINDINGS/credentials/${label}.txt"
    fi
done

grep -Ei 'salt\s*[:=]' "$STRINGS_DUMP" 2>/dev/null | sort -u | head -20 > "$FINDINGS/credentials/salts.txt" 2>/dev/null || true
SC=$(wc -l < "$FINDINGS/credentials/salts.txt" 2>/dev/null || echo 0)
if [[ "$SC" -gt 0 ]]; then
    echo "  salts: $SC"
    echo "### password salts" >> "$REPORT"
    echo '```' >> "$REPORT"
    cat "$FINDINGS/credentials/salts.txt" >> "$REPORT"
    echo '```' >> "$REPORT"
    echo "" >> "$REPORT"
fi

# --- network ---

echo "[*] scanning network config..."

{
    echo "## Network"
    echo ""
} >> "$REPORT"

```

```

grep -oE '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' "$STRINGS_DUMP" 2>/dev/null | sort -u | grep -vE
'^([0]\.0\0\0|127\0\0\0|[01]{255}\.0\0\0)' | head -100 > "$FINDINGS/configs/ip_addresses.txt"
IPC=$(wc -l < "$FINDINGS/configs/ip_addresses.txt")
echo "    $IPC unique IPs"
echo "### IP addresses ($IPC)" >> "$REPORT"
echo "    " >> "$REPORT"
cat "$FINDINGS/configs/ip_addresses.txt" >> "$REPORT"
echo "    " >> "$REPORT"
echo "" >> "$REPORT"

grep -oEi 'https?://[^\s\'<>]+' "$STRINGS_DUMP" 2>/dev/null | sort -u | head -100 > "$FINDINGS/configs/urls.txt"
UC=$(wc -l < "$FINDINGS/configs/urls.txt")
if [[ "$UC" -gt 0 ]]; then
    echo "    $UC URLs"
    echo "### URLs ($UC)" >> "$REPORT"
    echo "    " >> "$REPORT"
    cat "$FINDINGS/configs/urls.txt" >> "$REPORT"
    echo "    " >> "$REPORT"
    echo "" >> "$REPORT"
fi

find "$ROOTFS" -type f \( -name "resolv.conf" -o -name "hosts" -o -name "hostname" -o -name "*.conf" -path "*/netplan/*" -o -
name "interfaces" \) -path "*/etc/*" 2>/dev/null | while read -r nc; do
    is_binary "$nc" && continue
    is_text "$nc" || continue
    seen "$nc" && continue
    echo "### $(basename "$nc")" >> "$REPORT"
    echo "    " >> "$REPORT"
    head -200 "$nc" >> "$REPORT" 2>/dev/null || true
    echo "    " >> "$REPORT"
    echo "" >> "$REPORT"
    cp "$nc" "$FINDINGS/configs/" 2>/dev/null || true
done

find "$ROOTFS" -type f \( -name "iptables*" -o -name "nftables*" -o -name "firewall*" -o -name "*.rules" -path "*/iptables/*"
\ ) 2>/dev/null | while read -r fw; do
    is_binary "$fw" && continue
    is_text "$fw" || continue
    seen "$fw" && continue
    echo "### firewall: $(basename "$fw")" >> "$REPORT"
    echo "    " >> "$REPORT"
    head -500 "$fw" >> "$REPORT" 2>/dev/null || true
    echo "    " >> "$REPORT"
    echo "" >> "$REPORT"
    cp "$fw" "$FINDINGS/configs/" 2>/dev/null || true
done

# --- binaries ---

echo "[*] analyzing binaries..."

{
    echo "## Binaries"
    echo ""
} >> "$REPORT"

find "$ROOTFS" -type f -executable 2>/dev/null | head -200 | while read -r bin; do
    file "$bin" 2>/dev/null | grep -q "ELF" || continue
    seen "$bin" && continue
    ARCH=$(file "$bin" | grep -oE '(ARM|MIPS|x86-64|x86|PowerPC|RISC-V|aarch64|i[3-6]86)' | head -1)

    DANGEROUS=""
    STROUT=$(strings -a "$bin" 2>/dev/null)
    for func in system popen execve strcpy strcat sprintf gets; do
        c=$(echo "$STROUT" | grep -cw "$func" 2>/dev/null || true)
        c=$(( c + 0 )) 2>/dev/null || c=0
        [[ "$c" -gt 0 ]] && DANGEROUS+="$func ($c) "
    done

    [[ -z "$DANGEROUS" ]] && continue

    LINE="- \`${basename "$bin"}\` [$ARCH]: $DANGEROUS"
    echo "    $(basename "$bin") [$ARCH]: $DANGEROUS"

    if command -v readelf &>/dev/null; then
        canary=$(readelf -s "$bin" 2>/dev/null | grep -c "__stack_chk_fail" 2>/dev/null || true)
        canary=$(( canary + 0 )) 2>/dev/null || canary=0
        pie=$(readelf -h "$bin" 2>/dev/null | grep -c "DYN" 2>/dev/null || true)
        pie=$(( pie + 0 )) 2>/dev/null || pie=0
        nx=$(readelf -l "$bin" 2>/dev/null | grep -c "GNU_STACK.*RW" 2>/dev/null || true)
        nx=$(( nx + 0 )) 2>/dev/null || nx=0

        missing=""
        [[ "$canary" -eq 0 ]] && missing+="NO_CANARY "
        [[ "$pie" -eq 0 ]] && missing+="NO_PIE "
        [[ "$nx" -eq 0 ]] && missing+="NO_NX "
        [[ -n "$missing" ]] && LINE+=" | missing: $missing"
    fi

    echo "$LINE" >> "$REPORT"
done
echo "" >> "$REPORT"

# --- startup scripts ---

echo "[*] scanning startup scripts..."

{
    echo "## Startup Scripts"
    echo ""
} >> "$REPORT"

```

```

find "$ROOTFS" -type f \( -path "*/init.d/*" -o -path "*/rc.d/*" -o -path "*/systemd/system/*.service" -o -name "rc.local" -o
-name "inittab" -o -name "crontab" -path "*/etc/*" \) 2>/dev/null | while read -r s; do
    is_text "$s" || continue
    seen "$s" && continue
    echo "- \ $s\ " >> "$REPORT"

    if grep -lEi '(password|passwd|secret|key|token)\s*[:=]' "$s" &>/dev/null; then
        echo "    - contains credential references" >> "$REPORT"
    fi
    if grep -lEi '(telnet|nc -l|socat|ncat).*listen' "$s" &>/dev/null; then
        echo "    - network listener (possible backdoor)" >> "$REPORT"
    fi
done

find "$ROOTFS" -type f \( -name "crontab" -o -path "*/cron.d/*" -o -path "*/cron.daily/*" -o -path "*/cron.hourly/*" \)
2>/dev/null | while read -r cr; do
    is_text "$cr" || continue
    seen "$cr" && continue
    echo "" >> "$REPORT"
    echo "### crontab: $(basename "$cr")" >> "$REPORT"
    echo "'''" >> "$REPORT"
    grep -vE '\s*#\s*' "$cr" >> "$REPORT" 2>/dev/null || true
    echo "'''" >> "$REPORT"
done
echo "" >> "$REPORT"

# --- debug/backdoor ---

echo "[*] checking debug/backdoor indicators..."

{
    echo "## Debug and Backdoor Indicators"
    echo ""
} >> "$REPORT"

TREF=$(grep -ciE 'telnet.*(127\.0\.0\.1|localhost|0\.0\.0\.0)' "$STRINGS_DUMP" 2>/dev/null || echo 0)
if [[ "$TREF" -gt 0 ]]; then
    echo "### debug consoles" >> "$REPORT"
    echo "'''" >> "$REPORT"
    grep -Ei 'telnet.*(127\.0\.0\.1|localhost|0\.0\.0\.0)' "$STRINGS_DUMP" 2>/dev/null | sort -u | head -20 >> "$REPORT"
    echo "'''" >> "$REPORT"
    echo "" >> "$REPORT"
fi

for pat in "backdoor" "master.key" "master_password" "super.admin" "superadmin" "debug_shell" "reverse.shell" "bind.shell"
"god.mode" "test.mode" "factory_reset" "hidden.user" "maintenance_mode"; do
    h=$(grep -ci "$pat" "$STRINGS_DUMP" 2>/dev/null || echo 0)
    [[ "$h" -gt 0 ]] && echo "- \ $pat\ : $h references" >> "$REPORT"
done

find "$ROOTFS" -type f \( -name "*.ovpn" -o -name "openvpn*" -o -name "vpn*" -o -path "*/openvpn/*" -o -name "wg*.conf" \)
2>/dev/null | while read -r vpn; do
    seen "$vpn" && continue
    echo "- VPN config: \ $vpn\ " >> "$REPORT"
    cp "$vpn" "$FINDINGS/configs/" 2>/dev/null || true
done
echo "" >> "$REPORT"

# --- web ---

echo "[*] scanning web interfaces..."

{
    echo "## Web Interfaces"
    echo ""
} >> "$REPORT"

find "$ROOTFS" -type f \( -name "httpd.conf" -o -name "nginx.conf" -o -name "lighttpd.conf" -o -name "apache2.conf" -o -name
"uhttpd*" -o -name "*.php" -path "*/www/*" -o -name "*.cgi" \) 2>/dev/null | head -50 | while read -r w; do
    seen "$w" && continue
    echo "- \ $w\ " >> "$REPORT"
    if echo "$w" | grep -qE '\.(php|cgi)$'; then
        for func in eval exec system passthru shell_exec popen proc_open; do
            grep -q "$func" "$w" 2>/dev/null && echo "    - uses $func()" >> "$REPORT"
        done
    fi
done

find "$ROOTFS" -type f \( -name "phpinfo*" -o -name "adminer*" -o -name "phpmyadmin*" -o -name "info.php" -o -name "test.php"
-o -name "debug.php" \) 2>/dev/null | while read -r d; do
    seen "$d" && continue
    echo "- debug file: \ $d\ " >> "$REPORT"
done
echo "" >> "$REPORT"

# --- metadata ---

echo "[*] firmware metadata..."

{
    echo "## Firmware Metadata"
    echo ""
} >> "$REPORT"

find "$ROOTFS" -type f \( -name "os-release" -o -name "lsb-release" -o -name "issue" -o -name "version" -o -name "buildinfo"
-o -name "BUILD_INFO" \) -path "*/etc/*" 2>/dev/null | while read -r osf; do
    is_text "$osf" || continue
    seen "$osf" && continue
    echo "### $(basename "$osf")" >> "$REPORT"
    echo "'''" >> "$REPORT"
    cat "$osf" >> "$REPORT" 2>/dev/null || true

```

```

    echo "" >> "$REPORT"
    echo "" >> "$REPORT"
done

find "$ROOTFS" -type f \( -name "config-*" -path "*/boot/*" -o -name ".config" \) 2>/dev/null | while read -r kc; do
    is_text "$kc" || continue
    seen "$kc" && continue
    echo "### kernel config" >> "$REPORT"
    echo "" >> "$REPORT"
    grep -E '^CONFIG_(SECURITY|SECCOMP|DEBUG|CRYPTO|KEYS|SELINUX|APPARMOR)' "$kc" 2>/dev/null | sort >> "$REPORT" || true
    echo "" >> "$REPORT"
    dc=$(grep -c '^CONFIG_DEBUG' "$kc" 2>/dev/null || echo 0)
    [[ "$dc" -gt 5 ]] && echo "- $dc kernel debug options enabled" >> "$REPORT"
    echo "" >> "$REPORT"
done

ARCH_INFO=$(find "$ROOTFS" -type f -executable 2>/dev/null | head -5 | while read -r b; do
    file "$b" 2>/dev/null | grep -oE '(ARM|MIPS|x86-64|aarch64|PowerPC|RISC-V|i[3-6]86)' | head -1
done | sort -u | head -1)
[[ -n "${ARCH_INFO:-}" ]] && echo "- architecture: $ARCH_INFO" >> "$REPORT"

# --- crypto ---
echo "[*] scanning crypto material..."

{
    echo ""
    echo "## Crypto Material"
    echo ""
} >> "$REPORT"

grep -Ei '(AES|DES|RSA|ECDSA|HMAC|SHA256|MD5)' "$STRINGS_DUMP" 2>/dev/null | grep -Ei '(key|secret|password|salt|iv|nonce)' |
sort -u | head -30 > "$FINDINGS/credentials/crypto_refs.txt"

CC=$(wc -l < "$FINDINGS/credentials/crypto_refs.txt")
if [[ "$CC" -gt 0 ]]; then
    echo "### crypto references ($CC)" >> "$REPORT"
    echo "" >> "$REPORT"
    cat "$FINDINGS/credentials/crypto_refs.txt" >> "$REPORT"
    echo "" >> "$REPORT"
    echo "" >> "$REPORT"
fi

find "$ROOTFS" -type f \( -name "*.pl2" -o -name "*.pfx" -o -name "*.jks" -o -name "*.bks" \) 2>/dev/null | while read -r ks;
do
    seen "$ks" && continue
    echo "- keystore: \"$ks\"" >> "$REPORT"
    cp "$ks" "$FINDINGS/keys/" 2>/dev/null || true
done

# --- interesting files ---
{
    echo ""
    echo "## Interesting Files"
    echo ""
} >> "$REPORT"

find "$ROOTFS" -type f \( -name "*.db" -o -name "*.sqlite" -o -name "*.sqlite3" -o -name "*.sql" -o -name "*.mdb" \)
2>/dev/null | while read -r db; do
    seen "$db" && continue
    sz=$(stat -c%s "$db" 2>/dev/null || stat -f%z "$db" 2>/dev/null)
    echo "- \"$(basename "$db")\" ($sz bytes)" >> "$REPORT"
done

find "$ROOTFS" -type f \( -name "*.conf" -o -name "*.cfg" -o -name "*.ini" -o -name "*.yaml" -o -name "*.yml" -o -name
"*.json" -o -name "*.xml" -o -name "*.properties" -o -name "*.env" -o -name ".env*" \) -size +0c 2>/dev/null | while read -r
cfg; do
    is_text "$cfg" || continue
    grep -lEi '(password|passwd|secret|key|token|credential)s*[:=]' "$cfg" &>/dev/null || continue
    seen "$cfg" && continue
    echo "- secrets in: \"$cfg\"" >> "$REPORT"
    cp "$cfg" "$FINDINGS/configs/" 2>/dev/null || true
done

# --- summary ---

KEYS_N=$(find "$FINDINGS/keys" -type f 2>/dev/null | wc -l)
CERTS_N=$(find "$FINDINGS/certs" -type f 2>/dev/null | wc -l)
CONF_N=$(find "$FINDINGS/configs" -type f 2>/dev/null | wc -l)
CRED_N=$(find "$FINDINGS/credentials" -type f -size +0c 2>/dev/null | wc -l)
HASH_N=$(wc -l < "$FINDINGS/hashes/crackable.txt" 2>/dev/null || echo 0)

cat >> "$REPORT" <<EOF

---

## Summary

| category | count |
|-----|-----|
| private keys | $KEYS_N |
| certificates | $CERTS_N |
| config files | $CONF_N |
| credential files | $CRED_N |
| crackable hashes | $HASH_N |
| extracted files | $FILE_COUNT |
| strings indexed | $STR_COUNT |
EOF

# --- terminal summary ---

```

```

# collect squashfs image name from binwalk scan (first match, if any)
SQUASHFS_NAME=$(grep -i squashfs "$OUTDIR/binwalk_scan.txt" 2>/dev/null | grep -oE '[A-Za-z0-9_-]+\+squash[A-Za-z0-9_-]*' | head -1 || true)

# collect dir/symlink counts from the extracted tree
DIR_COUNT=$(find "$EXTRACT" -type d 2>/dev/null | wc -l | tr -d ' ')
SYM_COUNT=$(find "$EXTRACT" -type l 2>/dev/null | wc -l | tr -d ' ')

# binwalk version (graceful fallback)
BW_VER=$(binwalk --help 2>&1 | grep -oE 'Binwalk v[0-9]+\.[0-9]+' | head -1 2>/dev/null || binwalk --version 2>/dev/null | grep -oE '[0-9]+\.[0-9]+\.[0-9]+' | head -1 || echo "unknown")
[[ "$BW_VER" != "*" && "$BW_VER" != "unknown" ]] && BW_VER="v$BW_VER"

# credential match total across all pattern files
CRED_MATCH_TOTAL=0
for f in "$FINDINGS/credentials/"*.txt; do
    [[ -f "$f" ]] || continue
    n=$(wc -l < "$f" 2>/dev/null || echo 0)
    CRED_MATCH_TOTAL=$(( CRED_MATCH_TOTAL + n ))
done

echo ""
echo "=====
echo "[*] Extracting      : $FW_NAME"
echo "[*] MIME type       : $MIME"
if [[ -n "${BW_VER:-}" && "$BW_VER" != "unknown" ]]; then
    echo "[*] Running binwalk $BW_VER (recursive, depth=5) ..."
fi
if [[ -n "${SQUASHFS_NAME:-}" ]]; then
    echo "[*] SquashFS image  : $SQUASHFS_NAME"
    echo "[*] Running unsquashfs ..."
fi
echo "[*] Extracted      : $FILE_COUNT files, $DIR_COUNT dirs, $SYM_COUNT symlinks"
echo "[*] Strings indexed : $STR_COUNT tokens"
echo "-----
echo "[*] Private keys   : $KEYS_N"
echo "[*] Certificates    : $CERTS_N"
echo "[*] Config files    : $CONF_N"
echo "[*] Credential hits : $CRED_MATCH_TOTAL"
echo "[*] Crackable hashes: $HASH_N"
echo "-----
echo "[*] Analysis complete. See $REPORT"
echo "=====

```

Appendix B - Sample REPORT.md output (Askey 9361 pilot study)

```
# Firmware Security Analysis

**File:** BSR-04.03.70.a.2.aky.tgz
**Size:** 125376373 bytes (120M)
**Date:** 2026-05-19 11:50 UTC
**SHA256:** cbc566a4be3491a8f2ae0dc9df97a0ade4ef9e223af696415bdb463ff966b1be

---

## Extraction

- files: 4975
- type: application/gzip

## SSH Keys

- **PRIVATE KEY:** `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/init.d/dummy.key`
  Private-Key: (2048 bit, 2 primes)
  modulus:
    00:ca:9b:d0:69:71:63:ea:48:fc:61:e6:f6:0e:d3:
  - **PRIVATE KEY:** `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/sftpkeys/id_dsa`
  Private-Key: (1024 bit, 2 primes)
  modulus:
    00:99:5e:3c:79:ce:42:19:f6:b5:77:c8:d5:9e:70:
  - public key: `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/ssh/lab.pub`
  - public key: `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/sftpkeys/id_dsa.pub`

### sshd_config
```
Protocol 2
PermitRootLogin no
PasswordAuthentication no
PermitEmptyPasswords no
UsePAM yes
ClientAliveInterval 900
ClientAliveCountMax 0
Banner /etc/warning_preauth2
Subsystem sftp /usr/libexec/sftp-server
```

## Certificates

- `curl-ca-bundle.crt`: C=US, ST=Utah, L=Salt Lake City, O=Xcert EZ by DST, CN=Xcert EZ by DST, emailAddress=ca@digsigtrust.com
  - issuer: C=US, ST=Utah, L=Salt Lake City, O=Xcert EZ by DST, CN=Xcert EZ by DST, emailAddress=ca@digsigtrust.com
- IPsec: `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/ipsec.secrets`

## User Accounts

### passwd
```
root:x:0:0:root:/root:/bin/bash
primuser:x:500:500:Linux User,,,:/bin/bash
sshd:*:18:19:sshd:/var/run/sshd:/bin/sh
femtoAdmin:x:0:0:Femto Admin:/root:/bin/bash
factoryAdmin:x:0:0:Factory Admin:/root:/bin/bash
localAdmin:x:0:0:Local Admin:/root:/bin/bash
femtoOperator:x:500:500:Femto Operator,,,:/bin/bash
factoryOperator:x:500:500:Factory Operator,,,:/bin/bash
localOperator:x:500:500:Local Operator,,,:/bin/bash
```
- root-equivalent: `femtoAdmin` (uid 0, /bin/bash)
- root-equivalent: `factoryAdmin` (uid 0, /bin/bash)
- root-equivalent: `localAdmin` (uid 0, /bin/bash)

### shadow (asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/shadow.dir/shadow)
- `factoryAdmin`: `$1$FV[REDACTED]6g4q...`
- `factoryOperator`: `$1$nE[REDACTED]BmOBo...`
- `localAdmin`: `$1$tQ[REDACTED]aw/6...`
- `localOperator`: `$1$2[REDACTED]mzLx2ymG...`

### shadow (asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/shadow.field/shadow)

## Hardcoded Credentials

### secrets (3)
```
#RADIUS_SECRET="K+VzIDq7[REDACTED]jyCU"
```

### passwords (30)
```
DHCP_PPPOE_PASSWORD=${GREP} STIP.PPPoEPassword ${DHCPLEASE_FILE} | ${TR} -s ' ' | ${CUT} -f 4 -d ' '
getPassword::passwd is NULL \n
getPassword: passwd= %s %p \n
L&d - challengePassword: ASN.1 object has an invalid length
L&d - challengePassword: ASN.1 object is not a character string
L&d - challengePassword: ASN.1 object smaller than 2 octets
PASSWORD=${GREP} STIP.PPPoEPassword ${DHCPLEASE_FILE} | ${GREP} \``
Password:: %s
Password=[Z]
Set for MANAGEMENT_SERVER_PASSWORD, Device.ManagementServer.Password = fbsrpass failed in OAMTR
```

### password salts
```

```

    extract_salt: bad encoded salt length %u
extract_salt: expected salt len %d, got %d
extract_salt: invalid magic identifier
extract_salt: missing salt termination character
extract_salt: salt decode error
extract_salt: string too short
    
```

Network

IP addresses (75)

```

    04.00.10.00
    10.0.0.0
    10.1.5.1
    10.2.1.1
    1.10.140.2
    1.105.8.7
    1.108.12.6
    1.11.16.6
    1.117.8.13
    1.11.84.3
    1.12.116.1
    1.12.116.3
    1.1.272.2
    1.1.272.3
    1.13.140.1
    1.143.2.17
    1.16.140.1
    1.2.0.4
    1.2.1.3
    1.2.140.1
    1.2.140.3
    12.34.56.78
    12.34.56.99
    1.26.116.3
    127.127.1.0
    128.0.0.0
    1.28.116.1
    1.30.116.5
    1.30.140.7
    1.3.140.1
    135.86.[REDACTED]
    135.86.[REDACTED]
    1.40.32.3
    1.4.116.1
    1.4.140.1
    1.45.2.5
    1.48.116.4
    1.50.140.3
    1.51.2.4
    153.19.[REDACTED]
    1.57.76.1
    1.58.14.9
    1.59.16.1
    1.61.12.8
    1.6.140.1
    1.6.2.1
    1.66.116.1
    1.6.76.1
    1.67.76.5
    1.69.76.1
    1.7.140.1
    172.16.0.0
    1.8.116.1
    1.8.140.1
    1.9.116.3
    192.0.2.0
    192.0.2.1
    192.168.0.0
    192.168.0.1
    192.168.1.202
    192.168.1.211
    192.168.15.13
    192.168.16.13
    192.168.16.55
    192.168.1.99
    192.168.200.1
    192.168.200.4
    192.168.200.9
    192.168.201.0
    192.168.76.1
    192.168.76.14
    1.98.2.23
    2.1.8.7
    2.5.2.02
    2.6.29.6
    
```

URLs (41)

```

    http://%
http://135.86.[REDACTED]/bintro
http://bug
http://curl.haxx.
http://curlm.haxx.
http://dmalloc.com/
http://dmalloc.com/ $
http://e2f
http://etherx.jabber.org/
http://f

```

```

http://HNM.%
http://nw
http://procp
https://aco
https://HNM.%
https://www.i
https://www.tru
http://www.
http://www4.informatik.uni-erlangen.de/%7ekardel
http://www.c
http://www.core-
http://www.entru
http://www.f
http://www.gnu.org/
http://www.gnu.org/licen
http://www.i
http://www.iana.org/a
http://www.lib
http://www.libexpat.org
http://www.open
http://www.tcpcdump.org.
http://www.ti.com/
http://www.tru
http://www.valicert.com//Email=info@valicert.com
http://www.w3.org/*
http://www.w3.org/2000/xmln
http://www.w3.org/2001/XML
http://www.w3.org/2003/05/
http://www.w3.org/TR/REC-xml
http://www.w3.org/*XML
http://www.w3.org/XML/1998/name
...

### hosts
...
127.0.0.1      localhost.localdomain  localhost
...

### hostname
...
femtohsr_alu``

### firewall: firewall_recent
...
# kernel2
# Definitions of iptables "recent" interface for kernel2 and iptables V1.4.6
IPTRECENT_ADD="+
IPTRECENT_CLR="/"
IPTRECENT_LOCALDNSSERVERS="/proc/net/xt_recent/LOCALDNSSERVERS"
...

## Binaries
- `libprcl1.so` [MIPS]: system(3) popen(1) strcpy(1) strcat(1) | missing: NO_CANARY NO_NX
- `libprcl1_ns_shared.so` [MIPS]: strcat(1) sprintf(1) | missing: NO_CANARY NO_NX
- `programASR` [MIPS]: system(1) strcat(1) sprintf(1) | missing: NO_CANARY NO_PIE NO_NX
- `libnss_nisplus-2.5.so` [MIPS]: strcpy(1) | missing: NO_CANARY NO_NX
- `libproc-3.2.7.so` [MIPS]: strcpy(1) sprintf(1) | missing: NO_CANARY NO_NX
- `libutil-2.5.so` [MIPS]: strcpy(1) | missing: NO_CANARY NO_NX
- `libmudflap.so.0.0.0` [MIPS]: system(1) popen(3) execve(5) strcpy(2) strcat(2) sprintf(2) gets(1) | missing: NO_CANARY NO_NX
- `libpamc.so.0.82.1` [MIPS]: strcpy(1) sprintf(1) | missing: NO_CANARY NO_NX
- `ld-2.5.so` [MIPS]: system(2) | missing: NO_CANARY NO_NX
- `libnss_dns-2.5.so` [MIPS]: sprintf(1) | missing: NO_CANARY NO_NX
- `libpamc.so.0.82.2` [MIPS]: strcpy(1) | missing: NO_CANARY NO_NX
- `libnss_nis-2.5.so` [MIPS]: strcpy(1) sprintf(1) | missing: NO_CANARY NO_NX
- `libdl-2.5.so` [MIPS]: strcpy(1) | missing: NO_CANARY NO_NX
- `libstdc++.so.6.0.9` [MIPS]: strcpy(1) strcat(1) | missing: NO_CANARY NO_NX
- `libnsl-2.5.so` [MIPS]: system(1) strcpy(1) | missing: NO_CANARY NO_NX
- `libnss_compat-2.5.so` [MIPS]: strcpy(1) | missing: NO_CANARY NO_NX
- `libm-2.5.so` [MIPS]: sprintf(1) | missing: NO_CANARY NO_NX
- `libnss_files-2.5.so` [MIPS]: strcpy(1) | missing: NO_CANARY NO_NX
- `libmudflapth.so.0.0.0` [MIPS]: system(1) popen(3) execve(5) strcpy(2) strcat(2) sprintf(2) gets(1) | missing: NO_CANARY NO_NX
- `libpamc.so.0.83.0` [MIPS]: strcpy(1) | missing: NO_CANARY NO_NX
- `libnss_hesiod-2.5.so` [MIPS]: strcpy(1) | missing: NO_CANARY NO_NX
- `libc-2.5.so` [MIPS]: system(8) strcpy(1) strcat(1) gets(3) | missing: NO_CANARY NO_NX
- `pam_limits.so` [MIPS]: system(1) strcpy(1) | missing: NO_CANARY NO_NX
- `pam_mkhome.so` [MIPS]: execve(1) | missing: NO_CANARY NO_NX
- `pam_timestamp.so` [MIPS]: strcpy(1) | missing: NO_CANARY NO_NX
- `pam_unix.so` [MIPS]: system(1) execve(2) strcpy(1) | missing: NO_CANARY NO_NX
- `pam_namespace.so` [MIPS]: strcpy(1) | missing: NO_CANARY NO_NX
- `pam_fbsr_loginfailalarm.so` [MIPS]: system(1) | missing: NO_CANARY NO_NX
- `pam_issue.so` [MIPS]: strcat(1) | missing: NO_CANARY NO_NX
- `pam_pwhistory.so` [MIPS]: system(1) | missing: NO_CANARY NO_NX
- `pam_xauth.so` [MIPS]: strcpy(1) strcat(1) | missing: NO_CANARY NO_NX
- `pam_exec.so` [MIPS]: execve(2) | missing: NO_CANARY NO_NX
- `pam_radius_auth.so` [MIPS]: strcpy(1) | missing: NO_CANARY NO_NX
- `pam_filter.so` [MIPS]: strcpy(1) | missing: NO_CANARY NO_NX
- `pam_listfile.so` [MIPS]: strcpy(1) | missing: NO_CANARY NO_NX
- `pam_rhosts.so` [MIPS]: system(1) | missing: NO_CANARY NO_NX
- `pam_env.so` [MIPS]: strcpy(1) strcat(1) | missing: NO_CANARY NO_NX
- `libresolv-2.5.so` [MIPS]: strcpy(1) sprintf(1) | missing: NO_CANARY NO_NX
- `ss` [MIPS]: popen(1) strcpy(1) strcat(1) sprintf(1) | missing: NO_CANARY NO_PIE NO_NX
- `ip` [MIPS]: strcpy(1) strcat(1) sprintf(1) | missing: NO_CANARY NO_PIE NO_NX
- `mklost+found` [MIPS]: strcat(1) sprintf(1) | missing: NO_CANARY NO_PIE NO_NX
- `tcpdump` [MIPS]: system(5) sprintf(1) | missing: NO_CANARY NO_PIE NO_NX
- `ifstat` [MIPS]: sprintf(1) | missing: NO_CANARY NO_PIE NO_NX
- `rtmon` [MIPS]: strcat(1) sprintf(1) | missing: NO_CANARY NO_PIE NO_NX
- `rtacct` [MIPS]: sprintf(1) | missing: NO_CANARY NO_PIE NO_NX
- `tc` [MIPS]: strcpy(1) strcat(1) sprintf(1) | missing: NO_CANARY NO_PIE NO_NX

```

```

- `nstat` [MIPS]: sprintf(1) | missing: NO_CANARY NO_PIE NO_NX
- `genl` [MIPS]: strcat(1) sprintf(1) | missing: NO_CANARY NO_PIE NO_NX
- `dhclient` [MIPS]: system(1) popen(1) execve(2) strcpy(1) strcat(1) sprintf(1) | missing: NO_CANARY NO_PIE NO_NX
- `lnstat` [MIPS]: strcpy(1) strcat(1) | missing: NO_CANARY NO_PIE NO_NX
- `libip4tc.so.0.0.0` [MIPS]: strcpy(1) | missing: NO_CANARY NO_NX
- `libcrypto.so.0.9.8` [MIPS]: system(2) strcpy(2) strcat(2) sprintf(2) gets(2) | missing: NO_CANARY NO_NX
- `libebt_uolog.so` [MIPS]: strcpy(1) | missing: NO_CANARY NO_NX
- `libgmp.so.3.4.1` [MIPS]: strcpy(1) | missing: NO_CANARY NO_NX
- `pppoatm.so` [MIPS]: system(1) sprintf(1) | missing: NO_CANARY NO_NX
- `radius.so` [MIPS]: strcpy(1) strcat(1) sprintf(1) | missing: NO_CANARY NO_NX
- `passwordfd.so` [MIPS]: strcpy(1) | missing: NO_CANARY NO_NX
- `rp-pppoe.so` [MIPS]: system(1) strcpy(1) sprintf(1) | missing: NO_CANARY NO_NX
- `passprompt.so` [MIPS]: sprintf(1) | missing: NO_CANARY NO_NX
- `winbind.so` [MIPS]: sprintf(1) | missing: NO_CANARY NO_NX
- `ppp2tp.so` [MIPS]: sprintf(1) | missing: NO_CANARY NO_NX
- `libebt_log.so` [MIPS]: strcpy(1) | missing: NO_CANARY NO_NX
- `libdmallocalxx.so` [MIPS]: strcpy(2) strcat(2) | missing: NO_CANARY NO_NX
- `libcurl.so.4.3.0` [MIPS]: strcpy(1) | missing: NO_CANARY NO_NX
- `libcharon.so.0.0.0` [MIPS]: popen(1) strcpy(1) sprintf(1) | missing: NO_CANARY NO_NX
- `libbfd-2.16.1.so` [MIPS]: system(2) strcpy(1) strcat(1) sprintf(1) | missing: NO_CANARY NO_NX
- `libcurl.so.4.1.0` [MIPS]: strcpy(1) | missing: NO_CANARY NO_NX
- `libebt_nflog.so` [MIPS]: strcpy(1) | missing: NO_CANARY NO_NX
- `libz.so.1.2.3` [MIPS]: strcpy(1) strcat(1) sprintf(1) | missing: NO_CANARY NO_NX
- `libstrongswan.so.0.0.0` [MIPS]: popen(1) | missing: NO_CANARY NO_NX
- `libdmallocalth.so` [MIPS]: strcpy(2) strcat(2) | missing: NO_CANARY NO_NX
- `libssh2.so.1.0.1` [MIPS]: sprintf(1) | missing: NO_CANARY NO_NX
- `libtables.so.4.0.0` [MIPS]: strcpy(1) strcat(1) sprintf(1) | missing: NO_CANARY NO_NX
- `libpopt.so.0.0.0` [MIPS]: strcpy(1) strcat(1) | missing: NO_CANARY NO_NX
- `libebtc.so` [MIPS]: strcpy(1) sprintf(1) | missing: NO_CANARY NO_NX
- `libssl.so.0.9.8` [MIPS]: strcpy(2) | missing: NO_CANARY NO_NX
- `libdmallocal.so` [MIPS]: strcpy(2) strcat(2) | missing: NO_CANARY NO_NX
- `libip6tc.so.0.0.0` [MIPS]: strcpy(1) | missing: NO_CANARY NO_NX
- `libtz.so` [MIPS]: strcat(1) sprintf(1) | missing: NO_CANARY NO_NX
- `libdmallocalthcxx.so` [MIPS]: strcpy(2) strcat(2) | missing: NO_CANARY NO_NX

```

Startup Scripts

```

- `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/rc.d/init.d/ebtables`
- `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/crontab`
- `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/init.d/rcS`
- `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/init.d/ntp`
- `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/init.d/klogd`
- `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/init.d/loadModules`
- `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/init.d/syslogd`
- `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/init.d/randomseed`
- `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/init.d/getConfigParam`
- contains credential references
- `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/init.d/factorySeGWCheck`
- `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/init.d/startgetty`
- `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/init.d/init.shutdown`
- `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/init.d/functions`
- `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/init.d/femto_hw`
- `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/init.d/forceReset`
- `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/init.d/formatPramfs`
- `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/init.d/createVar`
- contains credential references
- `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/init.d/norcerts`
- `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/init.d/mountAll`
- `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/init.d/lockdown`
- `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/init.d/genSSHHostKeys`
- contains credential references
- `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/init.d/radius`
- contains credential references
- `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/init.d/checkmobs`
- `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/init.d/factoryReset`
- `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/inittab`

```

Debug and Backdoor Indicators

debug consoles

```

...
alias console="telnet 127.0.0.1 3140"
alias sbsp="telnet 127.0.0.1 7900"
) | /usr/bin/telnet localhost ${CONSOLE_PORT} &
...

```

```

- `master.key`: 27 references
- `test_mode`: 111 references
- `factory_reset`: 54 references

```

Web Interfaces

Firmware Metadata

Crypto Material

crypto references (30)

```

...
0013S: Time value must be in universal coordinated time format and must end with 'Z'
%12sRSA Public Key: (%d bit)
1des-iv32
1des-iv64
-1 MD5-based password algorithm
22. [23]Jeff Johnson <jbj@chatham.usdesign.com> massive prototyping
%2d MD5 %s # MD5 key
access to copy from a designated place, then offering equivalent
Active multi-threaded service %s: descriptor not set
Active single-threaded service %s: no server running, descriptor not set
Active single-threaded service %s: server running, descriptor set
# (Add `md5' after the module name to enable MD5 passwords)

```

```

aes des sha1 sha2 md5 random x509 pubkey pkcs1 pgp dnskey pem fips-prf xcbc hmac gmp attr      kernel-netlink socket-default
      stroke      updown      resolve      eap-femto eap-femto-aka eap-femto-DS2460B eap-femto-identity
aes_driver
AES Driver: could not register interrupt handler (err=%d)
AES driver: timedout waiting for done interrupt
aes_init_key
AES_INIT_KEY
aes key setup failed
aes key size t
AES: key size %u not supported
AES_set_decrypt_key
AES_set_encrypt_key
AES_unwrap_key
AES_wrap_key
Allocation error in generating RSA key
AppMsgCom::T(recvdLowBlkRsp):Message received for unknown destination MO
-apr1      MD5-based password algorithm, Apache variant
ASN1_UNIVERSALSTRING
ASN1_UNIVERSALSTRING_free
```


Interesting Files


```

- secrets in: `asky-out/extracted/squashfs_rootfs_V4_EC_L5.4.3.70.a.0003/etc/nsswitch.conf`
---

```


Summary

category	count
private keys	4
certificates	1
config files	8
credential files	5
crackable hashes	4
extracted files	4975
strings indexed	1553916


```